

瑞芯微车载摄像头开发指南

文件标识：RK-KF-YF-791

发布版本：V1.0.1

日期：2024-06-21

文件密级：☐绝密 ☐秘密 ☐内部资料 ☒公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2024 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址：福建省福州市铜盘路软件园A区18号

网址：www.rock-chips.com

客户服务电话：+86-4007-700-590

客户服务传真：+86-591-83951833

客户服务邮箱：fae@rock-chips.com

前言

概述

本文档旨在详细介绍瑞芯微（Rockchip）车载摄像头中间件软件。该系统经过精心设计，以确保高效、安全和可靠性，满足汽车行业的严格标准。

产品版本

芯片名称	系统版本	内核版本
RK3588M	AAOS 12/14	Linux 5.10/6.1
RK3576M	AAOS 14	Linux 6.1

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	谢科迪 蔡文萍 蔡文忠 曾益清	2024-04-08	初版
V1.0.1	蔡文忠	2024-06-21	驱动添加多路RAW Sensor以及多分辨率等的支持
V1.0.2	蔡文萍	2024-12-26	增加 RVCAM 多系统支持

目录

瑞芯微车载摄像头开发指南

1. 系统介绍
 - 1.1 系统模块与框架
 - 1.2 安全与质量标准
 - 1.3 功能开发路线图
 - 1.4 开放代码合作与商务流程
2. SEDES 驱动开发指南
 - 2.1 Maxim Camera Serdes 驱动概述
 - 2.1.1 Maxim Camera Serdes驱动框图
 - 2.1.2 Maxim Camera Serdes 驱动目录结构
 - 2.1.3 Maxim Camera Serdes 驱动 config 宏配置
 - 2.1.4 Maxim Camera Serdes 驱动 DTS 配置文件
 - 2.2 Maxim Camera SerDes DTS 配置说明
 - 2.2.1 I2C Init Sequence 的 DTS 配置说明
 - 2.2.2 Deserializer Dummy Sensor 的 DTS 配置
 - 2.2.3 Deserializer Support Mode 的 DTS 配置
 - 2.2.4 Deserializer local device 的 DTS 配置
 - 2.2.4.1 Local device gmsl-links 的 DTS 配置
 - 2.2.4.2 Local device video-pipes 的 DTS 配置
 - 2.2.4.3 Local device mipi-txphys 的 DTS 配置
 - 2.2.4.4 Local device extra-init-squence 的 DTS 配置
 - 2.2.5 Deserializer Remote device的DTS配置
 - 2.2.5.1 Remote device serializer 的 DTS 配置
 - 2.2.5.2 Remote device sensor 的 DTS 配置
 - 2.3 Maxim Camera SerDes 近端解串器驱动
 - 2.3.1 Local Deserializer - maxim4c
 - 2.3.2 Local Deserializer - maxim2c
 - 2.4 Maxim Camera SerDes 远端摄像头驱动
 - 2.4.1 Remote Serializer
 - 2.4.2 Remote Sensor
 - 2.4.3 RAW 摄像头驱动适配
 - 2.4.3.1 Kconfig 和 Makefile 添加配置
 - 2.4.3.2 模块驱动名字不同
 - 2.4.3.3 MCLK 提供方式不同
 - 2.4.3.4 电源提供方式不同
 - 2.4.3.5 控制时序 不同
 - 2.4.3.6 chip id 检测位置不同
 - 2.4.3.7 probe 添加 serializer 绑定关联
 - 2.4.3.8 开关流接口适配
3. RVCAM 中间件
 - 3.1 RVCAM 介绍
 - 3.2 RVCAM API 说明
 - 3.2.1 初始化与终止
 - 3.2.2 查询输入信息
 - 3.2.3 查询诊断信息
 - 3.2.4 管理虚拟摄像头流

- 3.2.5 参数及配置管理
 - 3.2.6 BUFFER 队列管理
 - 3.3 RVCAM 配置文件说明
 - 3.4 RVCAM 插件扩展
 - 3.5 RVCAM 性能指标说明
 - 3.5.1 CPU 使用率
 - 3.6 RVCAM 调试功能说明
 - 3.7 RVCAM 示例 RVC 单摄应用
 - 3.8 RVCAM 示例 AVIM + EGL 应用
- 4. EVS
 - 4.1 EVS 模块划分与目录介绍
 - 4.1.1 Native 层目录介绍
 - 4.1.1.1 EVS APP
 - 4.1.1.2 EVS Manager
 - 4.1.1.3 EVS HAL
 - 4.1.2 JAVA层目录介绍
 - 4.1.3 Test APK
 - 4.2 EVS 和 VHAL (Vehicle HAL) 状态关系
 - 4.2.1 Native
 - 4.2.2 Java
 - 4.3 框架流程图
 - 4.3.1 Camera 数据框架流程图
 - 4.3.2 显示数据框架流程图
 - 4.4 EVS 产品配置说明
 - 4.4.1 EVS APP产品配置说明
 - 4.4.2 EVS HAL evs_configuration.xml配置说明
 - 4.4.3 EVS HAL HIDL 产品配置说明
 - 4.4.4 EVS HAL AIDL 产品配置说明
- 5. HAL3
 - 5.1 基本功能说明
 - 5.1.1 AutomotiveCamera HAL3 流程介绍
 - 5.1.2 AutomotiveCamera HAL3 Provider 版本
 - 5.1.3 AutomotiveCamera HAL3 Device 版本
 - 5.1.4 automotive_camera_config.xml 配置说明
 - 5.2 新增功能说明
 - 5.2.1 BUFFER MANAGEMENT 功能
 - 5.2.2 BUFFER 透传功能
 - 5.3 性能指标说明
 - 5.4 调试功能说明
 - 5.4.1 CAMERA TRACE 调试功能
 - 5.4.2 查看相机配置参数
 - 5.5 产品配置说明
 - 5.5.1 AutomotiveCamera HAL HIDL 实例配置说明
 - 5.5.2 AutomotiveCamera HAL AIDL 实例配置说明
 - 5.6 单应用多开、多应用多开示例
- 6. RVCAM 多系统支持
 - 6.1 eRPC 简介
 - 6.2 RVCAM 跨系统通信实现
 - 6.2.1 RVCAM remote 模块实现

- 6.2.2 RVCAM remote 接口说明
 - 6.2.3 共享 buffer
 - 6.3 单系统与双系统编译和配置说明
 - 6.3.1 编译选项
 - 6.3.2 Camera Provider HAL3 模式配置
- 7. FAQ
- 8. 缩略语

1. 系统介绍

1.1 系统模块与框架

RVCAM (Rockchip Vehicle Camera) 是 Rockchip 针对汽车行业开发的一套基于 Rockchip 芯片的摄像头中间件解决方案，目的在于托管摄像头数据流的采集和算法处理，以实现包括不限于如虚拟多摄多开等功能。其系统框图如下：

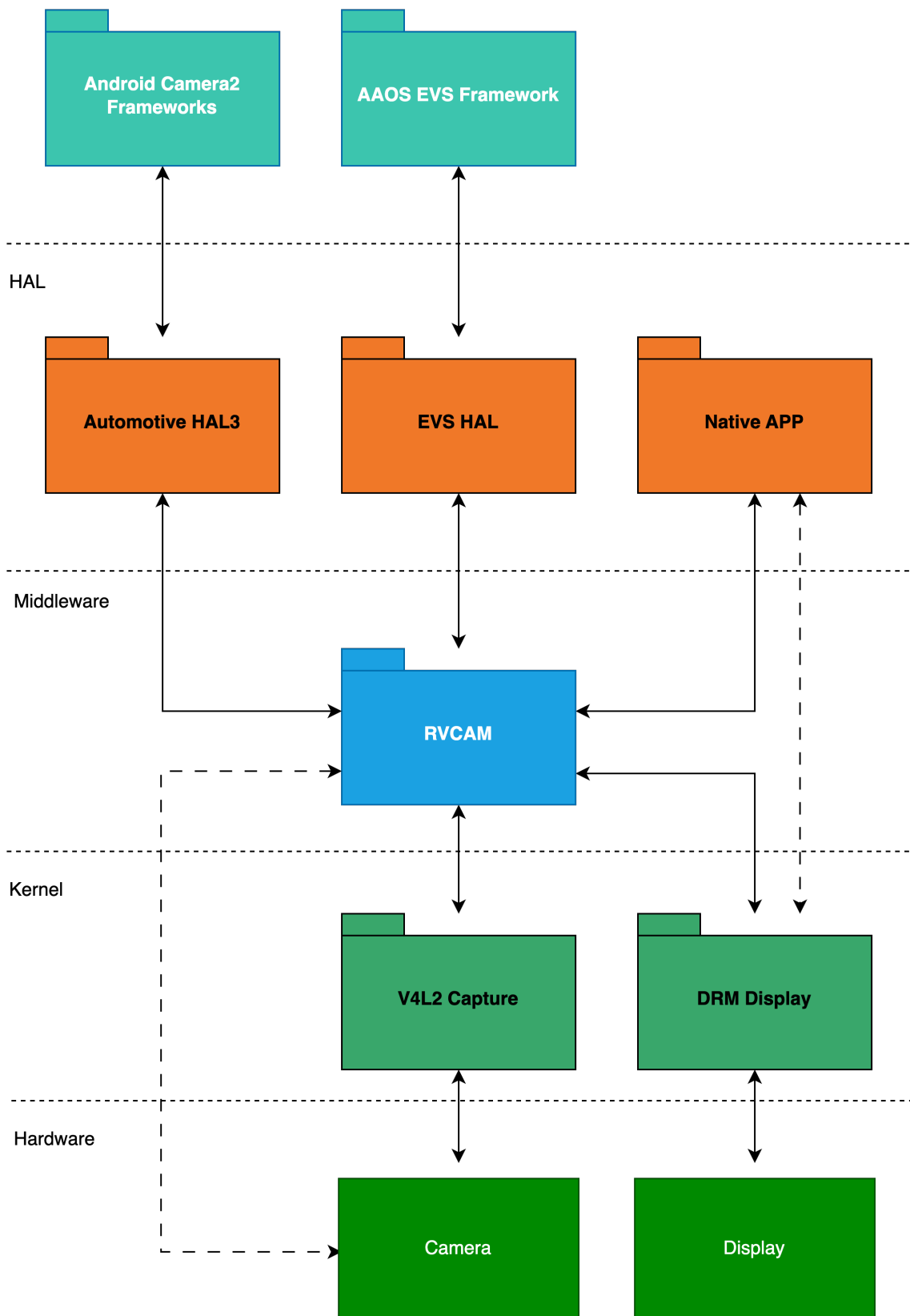


图 RVCAM 系统组件视图

基于 RVCAM 中间件，上层适配了 AOSP Camera2 API 的 Camera HAL3 接口的 Automotive Camera HAL3、适用于 AAOS 的 EVS HAL，以及允许 Vendor 厂商直接使用 RVCAM 的 API 开发 Native 应用程序，以实现最少依赖的快速倒车应用。

1.2 安全与质量标准

我们的开发流程参考并尽可能地遵循了 ISO26262 功能安全标准的流程，确保软件在整个生命周期中的安全性和可靠性。此外，我们还遵循以下行业标准和最佳实践：

- **编码规范:** 借鉴 MISRA-C、MISRA-C++ 并使用 Google C++ Lint 进行代码审查，以提高代码质量和减少潜在缺陷。
- **FuSa:** 以功能安全为前提，设计并实现符合 FuSa，确保系统在出现故障时能够以安全的方式响应。
- **FMEA:** 通过故障模式与影响分析（FMEA），我们积极收集客户反馈，并对车载摄像头系统可能遇到的故障情况进行预防和处理。

1.3 功能开发路线图

RVCAM 软件包的功能开发路线图规划了一系列比较常见的应用需求，以符合车载产品和客户的定制化需求。相关功能列举如下，优先级和顺序无关，我们会依据客户实际反馈进行调整。

- **YUV 摄像头虚拟多摄映射（完成）：**YUV 摄像头通过中间件可以实现使一个物理摄像头映射成多个摄像头，使系统框架能够多个应用打开。
- **SerDes 多路 RAW + ISP 支持（完成）：**支持通过 SerDes 接口传输多路原始图像数据，并集成图像信号处理器（ISP）。
- **单 SerDes 多 MIPI PHY 输出:** 通过单一 Des 芯片从多个 MIPI PHY 输出，满足客户不同 Sensor 使用 SoC 不同接口采集的需求。
- **车辆电源管理事件订阅:** 使摄像头系统能够响应 AAOS 车辆电源管理事件和策略，优化能耗。
- **摄像头热插拔支持:** 通过监控车辆运行过程中摄像头因抖动等因素产生的热插拔，在中间层处理响应，提供系统的鲁棒性和安全性。
- **低延时系统优化:** 对 Android 系统进行优化，以减少图像处理和传输的延迟。
- **ISP HAL 支持虚拟多摄:** 在传统 RAW Sensor 的 HAL 上新增支持虚拟多摄，允许框架多开。
- **摄像头诊断功能:** 实现摄像头状态监控和故障诊断功能，满足 ISO26262 功能安全的诊断需求。
- **第三方摄像头接口扩展:** 支持第三方虚拟摄像头接口，如 RTP 采集的视频，以实现自驾域、座舱域等的摄像头共享。

- **图像算法组件:** 以插件形式，集成自研和第三方的图像处理算法，如 ToF（Time of Flight）算法、QBC HDR Remosaic 算法，适应不同传感器应用的处理需求。
- **传感器融合框架:** 实现多摄像头融合和 IMU（惯性测量单元）等传感器数据的集成，方便实现如 BEV + Transform 之类算法，提高环境感知能力。
- **ISO17225 SOME/IP Camera Service:** 支持 ISO17225 标准的 SOME/IP Camera Service，确保与其他车辆系统的兼容性和互操作性。

1.4 开放代码合作与商务流程

中间件相关软件包版权归瑞芯微所有，采用闭源发布的方式。但是我们也鼓励开放源代码的合作，客户有需求可以向业务提出申请，沟通相关业务需求后，确定合作方式或者源代码释放的流程。

2. SEDES 驱动开发指南

2.1 Maxim Camera Serdes 驱动概述

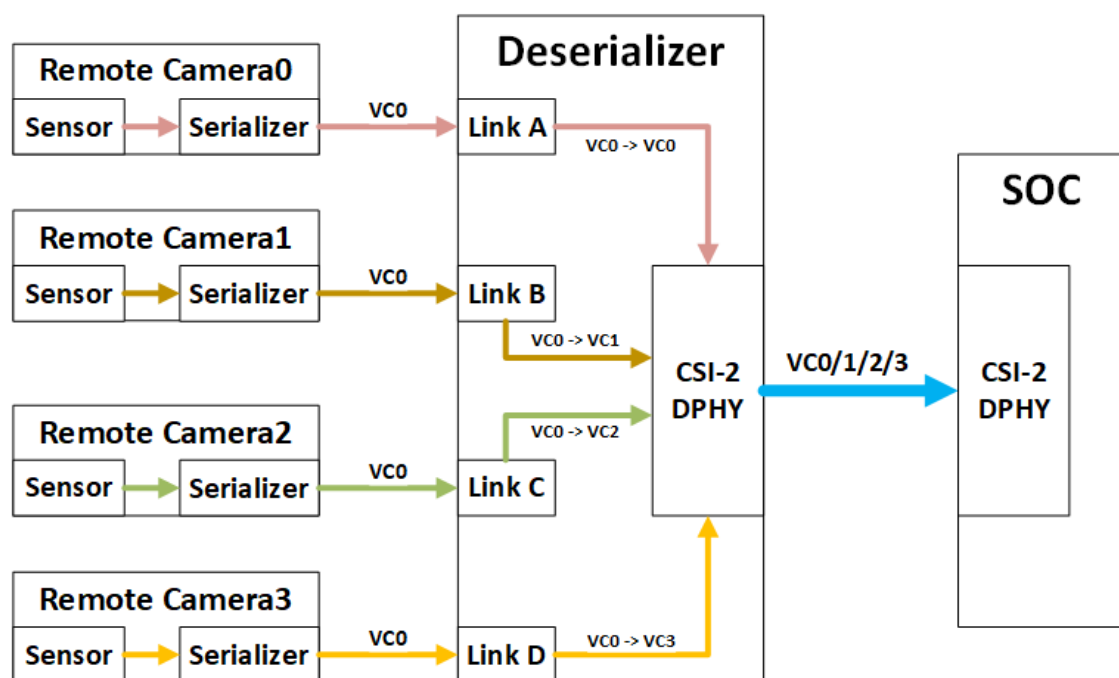
Maxim Camera SerDes V3 驱动是 Rockchip 针对美信 Camera SerDes 开发的驱动，目前支持的 SerDes 和 Sensor 型号如下表所示：

Local Deserializer	Remote Serializer	Remote Sensor
MAX96712 MAX96722 MAX96716 MAX96718	MAX9295 MAX96715 MAX96717	OV2311/OV2312 OX01F10 OX03J10 SC320AT OS04A10 dummy

远端的车载摄像头基本上都是将 serializer、sensor、eeprom、ISP 等集成一个整体的模组，通过 PoC 方式供电，具体的模组规格书需要咨询摄像头模组的供应商。一般情况下，如果YUV输出的摄像头模组，那么一般不太关注远端的 sensor 驱动，模组上电后从 eeprom 中加载配置数据并完成初始化工作。如果是RAW输出的摄像头模组，那么需要对远端的sensor进行初始化配置以及后续的曝光、增益等的配置。

2.1.1 Maxim Camera Serdes驱动框图

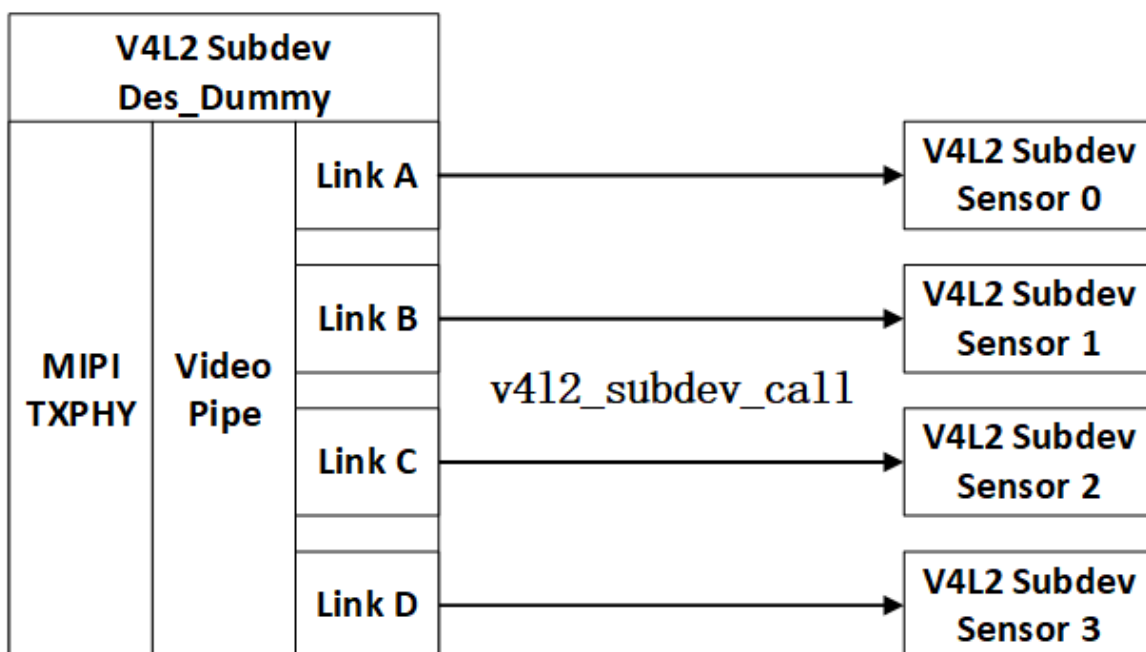
以 maxim 四条 link 的 maxim4c 驱动为例，SoC、Deserializer、Serializer 和 Sensor 的物理链路如下图所示：



远端四路摄像头通过线缆连接到近端的 Deserializer 的四个 Link 接口，远端摄像头的数据经过 Deserializer 的聚合映射成四路不同 VC 的 MIPI 数据，通过 CSI-2 DPHY 输送给 SoC 端。V3 版本驱动将近端 Deserializer 抽象成虚拟的 Sensor 摄像头来传输远端摄像头多路聚合的数据，实现 SoC 对远端四个摄像头的数据隔离，SerDes 和 SoC 的数据通路如下图所示：



近端 Deserializer 通过 `v4l2_subdev_call` 接口实现对远端的摄像头控制，通过 `s_power` 控制远端摄像头的上下电，通过 `s_stream` 控制远端摄像头的开关流，逻辑框下图所示：



2.1.2 Maxim Camera Serdes 驱动目录结构

Maxim Camera Serdes V3 驱动基于 kernel 的 media 框架，遵循 SerDes 远近端设备的设计理念，主体包含两大部分：local 设备和 remote 设备，驱动的目录结构：

```
kernel
|-----driver
|         |-----media
|         |         |-----i2c
|         |         |         |-----maxim
|         |         |         |         |-----Kconfig
|         |         |         |         |-----Makefile
|         |         |         |         |-----local
|         |         |         |         |         |-----Kconfig
|         |         |         |         |         |-----Makefile
|         |         |         |         |         |-----maxim2c
|         |         |         |         |         |         |-----Kconfig
|         |         |         |         |         |         |-----Makefile
|         |         |         |         |         |         |-----maxim2c_api.h
|         |         |         |         |         |         |-----maxim2c_compact.h
|         |         |         |         |         |         |-----maxim2c_drv.c
|         |         |         |         |         |         |-----maxim2c_drv.h
|         |         |         |         |         |         |-----maxim2c_i2c.c
|         |         |         |         |         |         |-----maxim2c_i2c.h
|         |         |         |         |         |         |-----maxim2c_link.c
|         |         |         |         |         |         |-----maxim2c_link.h
|         |         |         |         |         |         |-----maxim2c_mipi_txphy.c
|         |         |         |         |         |         |-----maxim2c_mipi_txphy.h
|         |         |         |         |         |         |-----maxim2c_pattern.c
|         |         |         |         |         |         |-----maxim2c_pattern.h
|         |         |         |         |         |         |-----maxim2c_remote.c
|         |         |         |         |         |         |-----maxim2c_v4l2.c
|         |         |         |         |         |         |-----maxim2c_video_pipe.c
|         |         |         |         |         |         |-----maxim2c_video_pipe.h
|         |         |         |         |         |-----maxim4c
|         |         |         |         |         |         |-----Kconfig
|         |         |         |         |         |         |-----Makefile
|         |         |         |         |         |         |-----maxim4c_api.h
|         |         |         |         |         |         |-----maxim4c_compact.h
|         |         |         |         |         |         |-----maxim4c_debugfs.c
|         |         |         |         |         |         |-----maxim4c_drv.c
|         |         |         |         |         |         |-----maxim4c_drv.h
|         |         |         |         |         |         |-----maxim4c_i2c.c
|         |         |         |         |         |         |-----maxim4c_i2c.h
|         |         |         |         |         |         |-----maxim4c_link.c
|         |         |         |         |         |         |-----maxim4c_link.h
|         |         |         |         |         |         |-----maxim4c_mipi_txphy.c
```

```

|-----maxim4c_mipi_txphy.h
|-----maxim4c_pattern.c
|-----maxim4c_pattern.h
|-----maxim4c_remote.c
|-----maxim4c_v4l2.c
|-----maxim4c_video_pipe.c
|-----maxim4c_video_pipe.h
|-----remote
|-----Kconfig
|-----Makefile
|-----max9295.c
|-----max96715.c
|-----max96717.c
|-----maxim_remote.h
|-----dummy.c
|-----ov231x.c
|-----ox01f10.c
|-----ox03j10.c
|-----sc320at.c
|-----os04a10.c

```

- ***maxim2c***

Camera SerDes 两条 Link 的近端 Deserializer 驱动，支持的芯片型号有 max96716、max96718 等。

- ***maxim4c***

Camera SerDes 四条 Link 的近端 Deserializer 驱动，支持的芯片型号有 max96712、max96722 等。

- ***remote***

Camera SerDes 远端设备的驱动，其中 Serializer 支持的芯片型号有 max9295、max96715、max96717等，Sensor 支持的芯片型号有 ov2311、ov2312、ox01f10、ox03j10、sc320at、os04a10 等，同时添加dummy sensor用来统一抽象远端摄像头，便以调试。

2.1.3 Maxim Camera Serdes 驱动 config 宏配置

Maxim Camera Serdes V3 驱动相关的内核 config 使能宏配置如下表所示：

功能说明	宏配置
Maxim Camera Serdes V3	CONFIG_VIDEO_MAXIM_SERDES=y
Local Deserializer	CONFIG_VIDEO_MAXIM_DES_MAXIM2C=y CONFIG_VIDEO_MAXIM_DES_MAXIM4C=y
Remote Serializer	CONFIG_VIDEO_MAXIM_SER_MAX9295=y CONFIG_VIDEO_MAXIM_SER_MAX96715=y CONFIG_VIDEO_MAXIM_SER_MAX96717=y
Remote Sensor	CONFIG_VIDEO_MAXIM_CAM_DUMMY=y CONFIG_VIDEO_MAXIM_CAM_SC320AT=y CONFIG_VIDEO_MAXIM_CAM_OX01F10=y CONFIG_VIDEO_MAXIM_CAM_OV231X=y CONFIG_VIDEO_MAXIM_CAM_OX03J10=y CONFIG_VIDEO_MAXIM_CAM_OS04A10=y

- **Maxim Camera SerDes V3版本驱动的总使能开关配置**

CONFIG_VIDEO_MAXIM_SERDES, 相应的 Kconfig 配置文件：

```
drivers/media/i2c/maxim/Kconfig
```

- **近端Deserializer的驱动使能配置**

CONFIG_VIDEO_MAXIM_DES_MAXIM2C：近端 Deserializer 两条 Link 的驱动。

CONFIG_VIDEO_MAXIM_DES_MAXIM4C：近端 Deserializer 四条 Link 的驱动。

相应的配置文件：

```
drivers/media/i2c/maxim/local/Kconfig
drivers/media/i2c/maxim/local/maxim2c/Kconfig
drivers/media/i2c/maxim/local/maxim4c/Kconfig
```

- **远端Serializer的驱动使能配置**

CONFIG_VIDEO_MAXIM_SER_MAX9295：远端 Serializer max9295 的驱动。

CONFIG_VIDEO_MAXIM_SER_MAX96715：远端 Serializer max96715 的驱动。

CONFIG_VIDEO_MAXIM_SER_MAX96717：远端 Serializer max96717 的驱动。

相应的配置文件：

```
drivers/media/i2c/maxim/remote/Kconfig
```

- **远端Sensor的驱动使能配置**

CONFIG_VIDEO_MAXIM_CAM_DUMMY：远端 Dummy Sensor 的驱动。

CONFIG_VIDEO_MAXIM_CAM_SC320AT：远端 Sensor Smartsens SC320AT 的驱动。

CONFIG_VIDEO_MAXIM_CAM_OX01F10：远端 Sensor OmniVision OX01F10 的驱动。

CONFIG_VIDEO_MAXIM_CAM_OV231X：远端 Sensor OmniVision OV2311/2 的驱动。

CONFIG_VIDEO_MAXIM_CAM_OX03J10：远端 Sensor OmniVision OX03J10 的驱动。

CONFIG_VIDEO_MAXIM_CAM_OS04A10：远端 Sensor OmniVision os04a10 的驱动。

相应的配置文件：

```
drivers/media/i2c/maxim/remote/Kconfig
```

2.1.4 Maxim Camera Serdes 驱动 DTS 配置文件

Maxim Camera Serdes V3 驱动 RK3588M Vehicle EVB 的 DTS 配置如下表所示：

Deserializer	DTS配置文件
MAX96712	rk3588-vehicle-evb-maxim-max96712-dcphy0.dtsi rk3588-vehicle-evb-maxim-max96712-dcphy1.dtsi rk3588-vehicle-evb-maxim-max96712-dphy0.dtsi rk3588-vehicle-evb-maxim-max96712-dphy3.dtsi rk3588-vehicle-evb-maxim-max96712-dphy3-dummy.dtsi rk3588-vehicle-evb-maxim-max96712-dphy3-os04a10.dtsi
MAX96722	rk3588-vehicle-evb-maxim-max96722-dcphy0.dtsi rk3588-vehicle-evb-maxim-max96722-dphy0.dtsi rk3588-vehicle-evb-maxim-max96722-dphy3.dtsi rk3588-vehicle-evb-maxim-max96722-dphy3-dummy.dtsi

2.2 Maxim Camera SerDes DTS 配置说明

以 rk3588-vehicle-evb-maxim-max96712-dphy3.dtsi 为例来说明 dts 的具体配置细节，重点描述 maxim4c 的模块配置部分。

远端多路raw sensor时，可以参考 rk3588-vehicle-evb-maxim-max96712-dphy3-os04a10.dtsi 的配置，通过宏 MULTI_SENSOR_LINK_TO_ISP 将ISP使能部分进行隔离。主要涉及的配置如下：

- serdes-local-device 节点添加属性 remote-routing-to-isp = <1>，使能多路 RAW Sensor 的 SerDes 相关驱动配置。
- max96712_dphy3_cam0 远端摄像头 port 添加 remote-endpoint 链接到 ISP 节点。

3. `rkcif_mipi_lvds4` 节点添加 `camera-over-bridge` 属性配置，指示 CIF 驱动远端多路 RAW Sensor 模式。

2.2.1 I2C Init Sequence 的 DTS 配置说明

Maxim SerDes 的寄存器配置分散且繁多，和 Sensor 类似，美信技术支持提供的配置基本上都是一系列的配置序列，所以为了方便相关配置的导入和扩展，Rockchip Camera SerDes 的 dts 配置中引入 i2c 的 init sequence，参考样例如下所示：

```
link-init-sequence {
    seq-item-size = <5>; // reg-addr-len + reg-val-len * 2 + 1
    reg-addr-len = <2>; // 1: 8bits, 2: 16bits
    reg-val-len = <1>; // 1: 8bits, 2: 16bits, 3: 24bits

    // reg_addr reg_val val_mask delay
    init-sequence = [
        14 D1 03 00 00 // VGAHiGain
        14 45 00 00 00 // Disable SSC
    ];
};
```

上面的序列是对一个 I2C 设备 16bits 寄存器地址写入 8bits 的数据，操作了两个寄存器：

- **14 D1 03 00 00** 表示寄存器地址 0x14D1 写入 0x03；
- **14 45 00 00 00** 表示寄存器地址 0x1445 写入 0x00。

I2C Init Sequence 的参数项说明如下：

- ***seq-item-size***

定义 init-sequence 的单个数据项的字节长度，即一组寄存器配置的数据字节长度。单个数据项的字节长度 = 寄存器地址的字节长度 + 寄存器数据的字节长度 + 寄存器数据对应 mask 的字节长度 + delay 延时的一个字节长度。

- ***reg-addr-len***

定义 init-sequence 设备的寄存器地址的字节长度，1 表示 8bits，2 表示 16bits，3 表示 24bits，4 表示 32bits。

- ***reg-val-len***

定义 init-sequence 写入寄存器数据的字节长度，1 表示 8bits，2 表示 16bits，3 表示 24bits，4 表示 32bits。

- ***init-sequence***

定义 I2C 设备的配置序列的一维数组字节数据，便于代码的阅读，一般书写时规定一组寄存器配置一行，注意数组每行数据的字节数和 seq-item-size 务必要一致，否则数据加载时会校验失败。

I2C Init Sequence 的数据结构:

```
/* seq_item_size = reg_len + val_len * 2 + 1 */
struct maxim4c_i2c_init_seq {
    struct maxim4c_i2c_regval *reg_init_seq;
    u32 reg_seq_size;
    u32 seq_item_size;
    u32 reg_len;
    u32 val_len;
};
```

I2C Init Sequence 的加载 API 操作函数:

```
int maxim4c_i2c_load_init_seq(struct device *dev,
    struct device_node *node, struct maxim4c_i2c_init_seq *init_seq);
```

I2C Init Sequence 的运行 API 操作函数:

```
int maxim4c_i2c_run_init_seq(struct i2c_client *client,
    struct maxim4c_i2c_init_seq *init_seq);
```

2.2.2 Deserializer Dummy Sensor 的 DTS 配置

Deserializer max96712 抽象成 dummy sensor 设备挂载在 i2c6 下, 通过 port 接口链接到 mipi_dphy3, sensor 详细的配置可以参考 sensor 驱动的技术文档, 这里不展开具体描述。

```
&i2c6 {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&i2c6m3_xfer>;

    max96712_dphy3: max96712@29 {
        compatible = "maxim4c,max96712";
        status = "okay";
        reg = <0x29>;
        clock-names = "xvclk";
        clocks = <&max96712_dphy3_osc0 0>;
        pinctrl-names = "default";
        pinctrl-0 = <&max96712_dphy3_errb>, <&max96712_dphy3_lock>;
        power-domains = <&power RK3588_PD_VI>;
        rockchip,grf = <&sys_grf>;
        vcc1v2-supply = <&max96712_dphy3_vcc1v2>;
        vcc1v8-supply = <&max96712_dphy3_vcc1v8>;
        pwn-supply = <&max96712_dphy3_pwn_regulator>;
    };
};
```

```

lock-gpios = <&gpio3 RK_PB4 GPIO_ACTIVE_HIGH>;

rockchip,camera-module-index = <0>;
rockchip,camera-module-facing = "back";
rockchip,camera-module-name = "default";
rockchip,camera-module-lens-name = "default";

port {
    max96712_dphy3_out: endpoint {
        remote-endpoint = <&mipi_dphy3_in_max96712>;
        data-lanes = <1 2 3 4>;
    };
};
};
};

```

2.2.3 Deserializer Support Mode 的 DTS 配置

由于 Camera SerDes 远端摄像头的多样性，配置多变，为了确保驱动独立简洁，这里将 dummy sensor 的 support mode 提取到 dts 里面配置，通过驱动加载是读取的方式来适配远端设备多样性的配置需求。

```

/* support mode config start */
support-mode-config {
    status = "okay";

    bus-format = <MEDIA_BUS_FMT_UYVY8_2X8>;
    sensor-width = <1920>;
    sensor-height = <1440>;
    max-fps-numerator = <10000>;
    max-fps-denominator = <300000>;
    bpp = <16>;
    link-freq-idx = <20>;
    vc-array = <0x10 0x20 0x40 0x80>; // VC0~3: bit4~7
};
/* support mode config end */

```

- **bus-format**

配置 sensor 的数据格式，具体参见 media-bus-format.h 定义。

- **sensor-width**

配置 sensor 的宽度。

- **sensor-height**

配置 sensor 的高度。

- **crop-rect**

配置 sensor crop 矩阵，格式： `crop-rect = <left top width height>`，配置样例如下：

```
crop-rect = <0 1 1920 1280>; // [ left, top, width, height ]
```

驱动会解析配置的矩阵并通过 `v4l2_subdev_pad_ops` 的 `get_selection` 传递给应用层。

- **max-fps-numerator**

配置 sensor 的最小帧率。

- **max-fps-denominator**

配置 sensor 的最大帧率。

- **bpp**

配置 sensor 的 BPP

- **link-freq-idx**

配置 sensor 的 MIPI clock索引。

- **vc-info**

配置 sensor 的多通道信息，用于支持多分辨率和格式情况，配置样例如下：

```
vc-info = <
    /* enable width height bus_fmt data_type data_bit */
    1 1920 1440 MEDIA_BUS_FMT_UYVY8_2X8 0 0 // vc0
    1 1920 1440 MEDIA_BUS_FMT_UYVY8_2X8 0 0 // vc1
    1 1920 1280 MEDIA_BUS_FMT_UYVY8_2X8 0 0 // vc2
    1 1920 1280 MEDIA_BUS_FMT_UYVY8_2X8 0 0 // vc3
>;
```

每路VC配置6个参数，顺序如下：enable width height bus_fmt data_type data_bit，上面例子中配置了4路VC，其中VC0和VC1一样，VC2和VC3一样，每路VC的参数配置细节如下：

- **enable**

第一个参数，VC通道使能配置。0：关闭；1：使能。

- **width**

第二个参数，VC通道的宽配置。

- **height**

第三个参数，VC通道的高配置。

- **bus_fmt**

第四个参数，VC通道的bus format配置。

- **data_type**

第五个参数，VC通道的data type配置。

- ***data_bit***

第六个参数，VC通道的data bit配置。

vicap 通过 v4l2 的 ioctl 命令 RKMODULE_GET_CHANNEL_INFO 来获取通道信息。

注意：vc-info一旦配置使能，那么 crop-rect 将会失效。目前不支持多通道模式的 crop-rect。

- ***vc-array***

配置 sensor 的 VC 通道，从 v3.06.00 版本开始，默认不配置，驱动会默认赋值，只有在VC通道需要特殊改动时才需要添加vc-array配置。

相关的配置可以参考 sensor 的驱动说明，这里不细化展开。函数

maxim4c_support_mode_init 实现 support mode 的 dts 解析。

```
static int maxim4c_support_mode_init(maxim4c_t *maxim4c)
```

2.2.4 Deserializer local device 的 DTS 配置

Local device 的配置由四个部分组成：gmsl-links、video-pipes、mipi-txphys 和 extra-init-sequence。其中gmsl-links 是 deserializer link 模块的相关配置，video-pipes 是 deserializer pipe 模块的相关配置，mipi-txphys 是 deserializer txphy 模块的相关配置，extra-init-sequence 用于扩展的配置序列，比如帧同步、GPIO 等的配置。

- ***remote-routing-to-isp***

配置 多路raw sensor，1：使能；0：关闭。参考配置如下：

```
remote-routing-to-isp = <1>; /* remote camera route to ISP */
```

remote-routing-to-isp 使能配置时，主要影响如下：

1. 远端摄像头的 s_power 和 s_stream 控制权由 deserializer start_stream 统一管理操作 变成了 cif驱动按通路管理操作。
2. maxim4c_ioctl 执行命令 RKMODULE_GET_CAPTURE_MODE 反馈 capture_info 的 mode 状态。

2.2.4.1 Local device gmsl-links 的 DTS 配置

gmsl-links 是 deserializer link 模块的配置，对应的模块代码是 maxim4c_link.c，解析函数：

```
int maxim4c_link_parse_dt(maxim4c_t *maxim4c, struct device_node *of_node)
```

gmsl-links 主要配置参数项：

- **link-vdd-ldo1-en**: 1: 使能, 0: 关闭。

如果 deserializer 的 VDD = 1.2V, 那么需要使能 REG_ENABLE 和 REG_MNL, 配置函数:

```
static int maxim4c_link_enable_vdd_ldo1(maxim4c_t *maxim4c)
```

- **link-vdd-ldo2-en**: 1: 使能, 0: 关闭。

```
static int maxim4c_link_enable_vdd_ldo2(maxim4c_t *maxim4c)
```

- **gmsl-link-config-0/1/2/3**

deserializer 四条 link 的配置, 这里以 gmsl-link-config-0 为例对参数说明。

```
// Link A: link-id = 0
gmsl-link-config-0 {
    status = "okay";
    link-id = <0>; // Link ID: 0/1/2/3

    link-type = <1>;
    link-rx-rate = <1>;
    link-tx-rate = <0>;

    link-remote-cam = <&max96712_dphy3_cam0>; // remote camera

    link-init-sequence {
        seq-item-size = <5>; // reg-addr-len + reg-val-len * 2 + 1
        reg-addr-len = <2>; // 1: 8bits, 2: 16bits
        reg-val-len = <1>; // 1: 8bits, 2: 16bits, 3: 24bits

        // reg_addr reg_val val_mask delay
        init-sequence = [
            14 D1 03 00 00 // VGAHiGain
            14 45 00 00 00 // Disable SSC
        ];
    };
};
```

参数配置细节如下：

- **link-id**

配置 link 的 ID, 对应数值: 0 - Link A、1 - Link B、2 - Link C、3 - Link D。

- **link-type**

配置 link 的类型, 对应数值: 0 - GMSL1, 1 - GMSL2。

- **link-rx-rate**

配置 link 的正向速率，对应数值：0 – 3Gbps, 1 – 6Gbps。

- **link-tx-rate**

配置 link 的反向速率，对应数值：0 - 187.5Mbps。

- **link-remote-cam**

配置 link 的远端 camera 节点的 phandle，对应的远端摄像头节点。

- **link-init-sequence**

配置 link 的 init sequence，根据对应的 link id 配置相应的寄存器组。

注意：link init sequence 数组具体的寄存器配置含义需要参照美信规格书。

2.2.4.2 Local device video-pipes 的 DTS 配置

video-pipes 是 deserializer video pipe 模块的配置，对应的模块代码是 maxim4c_video_pipe.c，其解析函数：

```
int maxim4c_video_pipe_parse_dt(maxim4c_t *maxim4c, struct device_node
*of_node)
```

video-pipes 主要的配置参数项：

- **video-pipe-config-0/1/2/3**

对应video pipe，一般是4条pipes，可以支持8条，具体根据实际项目需求配置。这里以 video-pipe-config-0为例对参数说明。

```
// Video Pipe 0
video-pipe-config-0 {
    status = "okay";
    pipe-id = <0>; // Video Pipe ID: 0/1/2/3/4/5/6/7

    pipe-idx = <2>; // Video Pipe X/Y/Z/U: 0/1/2/3
    link-idx = <0>; // Link A/B/C/D: 0/1/2/3

    pipe-init-sequence {
        seq-item-size = <5>; // reg-addr-len + reg-val-len * 2 + 1
        reg-addr-len = <2>; // 1: 8bits, 2: 16bits
        reg-val-len = <1>; // 1: 8bits, 2: 16bits, 3: 24bits

        // reg_addr reg_val val_mask delay
        init-sequence = [
            // Send YUV422, FS, and FE from Video Pipe 0 to Controller 1
            09 0B 07 00 00 // Enable 0/1/2 SRC/DST Mappings
            09 2D 15 00 00 // SRC/DST 0/1/2 -> CSI2 Controller 1;
```

```

        // For the following MSB 2 bits = VC, LSB 6 bits = DT
    09 0D 1e 00 00 // SRC0 VC = 0, DT = YUV422 8bit
    09 0E 1e 00 00 // DST0 VC = 0, DT = YUV422 8bit
    09 0F 00 00 00 // SRC1 VC = 0, DT = Frame Start
    09 10 00 00 00 // DST1 VC = 0, DT = Frame Start
    09 11 01 00 00 // SRC2 VC = 0, DT = Frame End
    09 12 01 00 00 // DST2 VC = 0, DT = Frame End
    ];
};
};

```

- ***pipe-id***

配置 pipe 的 ID，对应数值：0、1、2、3、4、5、6、7。

- ***pipe-index***

配置 pipe 输入的 ID，对应数值：0 – Pipe X，1 – Pipe Y，2 – Pipe Z，3 – Pipe U。

- ***link-index***

配置 pipe 输出的 link id，对应数值：0 – Link A、1 – Link B、2 – Link C、3 – Link D。

- ***pipe-init-sequence***

配置 pipe 的 init sequence，根据对应的 pipe id 配置相应的寄存器组。

注意： pipe init sequence 数组的具体配置比较复杂，和实际的远端摄像头要匹配，主要涉及 CSI2 Controller 的选择、VC 和 DT 的配置、Src 和 Dst 的 Mapping 等，具体的寄存器含义需要参照美信规格书以及相关的技术支持。

- ***parallel-mode-config***

配置 parallel mode 的 init sequence，应用于 GMSL1 Data 等需要使能 parallel mode 的场景，一般配置 VC software override、BPP、DT、YUV8-/10-bit mux mode 等。

2.2.4.3 Local device mipi-txphys 的 DTS 配置

mipi-txphys 是 deserializer mipi txphy 模块的配置，对应的模块代码是 maxim4c_mipi_txphy.c，其解析函数：

```

int maxim4c_mipi_txphy_parse_dt(maxim4c_t *maxim4c, struct device_node
*of_node)

```

mipi-txphys 主要的配置参数项：

- ***phy-mode***

配置 txphy 的工作模式，对应数值如下：

0 – 2X4Lanes, 1 – 4X2Lanes, 2 – 1X4Lanes_2X2Lanes, 3 – 2X2Lanes_1X4Lanes

- **phy-force-clock-out**

配置 mipi clock 强制输出，对应数值：0 – 关闭，1 – 打开。

注意：默认要使能，可以保证远端接入不完整的情况下 MIPI PHY 可以正常输出 clock。

- **phy-force-clk0-en**

配置 PHY0 和 PHY1 在 Combined 模式下的 Clock 从非 master 的 PHY0 输出，对应数值：
0 – Clock 从 PHY1 输出（默认），1 – Clock 从 PHY0 输出。

- **phy-force-clk3-en**

配置 PHY2 和 PHY3 在 Combined 模式下的 Clock 从非 master 的 PHY3 输出，对应数值：
0 – Clock 从 PHY2 输出（默认），1 – Clock 从 PHY3 输出。

- **mipi-txphy-config-0/1/2/3**

对应 txphy，和 phy-mode 匹配，一般情况下采用 PHY0 和 PHY1 在 Combined 模式。这里以 mipi-txphy-config-0 为例对参数说明。

```
// MIPI TXPHY A: phy-id = 0
mipi-txphy-config-0 {
    status = "okay";
    phy-id = <0>; // MIPI TXPHY ID: 0/1/2/3

    phy-type = <0>;
    auto-deskew = <0x80>;
    data-lane-num = <4>;
    data-lane-map = <0x4>;
    vc-ext-en = <0>;
};
```

- **phy-id**

配置 mipi txphy 的 ID，对应数值：0 – PHY0、1 – PHY1、2 – PHY2、3 – PHY3。

- **phy-type**

配置 mipi txphy 的 type，对应数值：0 – DPHY、1 – CPHY。

- **auto-deskew**

配置 mipi txphy 的 deskew，对应数值：0 – 关闭，其它数值配置参见美信规格书。

- **data-lane-num**

配置 mipi txphy 的 lane 数目，对应数值：1 ~ 4。

- **data-lane-map**

配置 mipi txphy 的 lane map，对应数值配置参见美信规格书。

- **vc-ext-en**

配置 mipi txphy 的 vc 拓展 bit 使能，对应数值：0 – 关闭，1 – 使能。

- **tunnel-enable**

配置 mipi txphy 的 tunnel 模式，对应数值：0 – 关闭，1 – 使能。

- **tunnel-vs-wait**

配置 mipi txphy 的 tunnel 模式需要过滤帧的数量，对应数值配置参见美信规格书。

2.2.4.4 Local device extra-init-sequence 的 DTS 配置

为了方便一些不确定性的配置和非模块化的配置，这里特地设计了一个可拓展的配置序列，用于诸如帧同步相关的配置、gpio的配置等，对应的加载和执行操作函数：

```
static int maxim4c_extra_init_seq_parse(maxim4c_t *maxim4c, struct
device_node *node)

static int maxim4c_run_extra_init_seq(maxim4c_t *maxim4c)
```

2.2.5 Deserializer Remote device的DTS配置

Maxim 的 Camera SerDes v3 版本驱动对远端设备进行了重构，将远端的 serializer 和 sensor 的 I2C 设备开放出来，sensor 使用标准的 v4l2 subdev，便以通过 v4l2_subdev_call 接口控制远端设备，同时使用 i2c-mux 将远端的 serializer 和 sensor 灵活绑定。关于 I2C-MUX 具体的技术细节可以参见 Linux 的文档说明，这里不做细化展开描述。I2C Mux DTS 节点定义的时候需要特别注意一点：serializer 节点的配置必须在 sensor 节点之前，以确保 serializer 驱动的 probe 优先于 sensor，那么 sensor 驱动 probe 的时候就可以正确绑定 serializer 设备，实现远端设备的灵活搭配。远端多设备的 i2c-mux dts 实例如下所示。

```
/* i2c-mux start */
i2c-mux {
    #address-cells = <1>;
    #size-cells = <0>;

    i2c@0 {
        #address-cells = <1>;
        #size-cells = <0>;
        reg = <0>;

        // Note: Serializer node defined before camera node
        max96712_dphy3_ser0: max96717@41 {
            compatible = "maxim,ser,max96717";
            reg = <0x41>;
        };

        max96712_dphy3_cam0: sc320at@31 {
```

```

        compatible = "maxim,smartsens,sc320at";
        reg = <0x31>;
    };
};

i2c@1 {
    #address-cells = <1>;
    #size-cells = <0>;
    reg = <1>;

    // Note: Serializer node defined before camera node
    max96712_dphy3_ser1: max96717@42 {
        compatible = "maxim,ser,max96717";
        reg = <0x42>;
    };

    // Note: Serializer node defined before camera node
    max96712_dphy3_cam1: sc320at@32 {
        compatible = "maxim,smartsens,sc320at";
        reg = <0x32>;
    };
};
};
/* i2c-mux end */

```

2.2.5.1 Remote device serializer 的 DTS 配置

Remote serializer 的 DTS 配置主要是 i2c 地址映射以及对应摄像头的 mclk、reset 信号等的配置，以max96712_dphy3_ser0 为例：

```

// Note: Serializer node defined before camera node
max96712_dphy3_ser0: max96717@41 {
    compatible = "maxim,ser,max96717";
    reg = <0x41>;

    ser-i2c-addr-def = <0x40>;

    ser-init-sequence {
        seq-item-size = <5>; // reg-addr-len + reg-val-len * 2 + 1
        reg-addr-len = <2>; // 1: 8bits, 2: 16bits
        reg-val-len = <1>; // 1: 8bits, 2: 16bits, 3: 24bits

        // reg_addr reg_val val_mask delay
        init-sequence = [
            03 02 10 00 00
            14 17 00 00 00
        ];
    };
};

```

```

        14 32 7f 00 00
    ];
};
};

```

- **max96717@41**

配置 serializer max96717 节点，i2c 地址使用的是映射后的 7 位新地址。

- **ser-i2c-addr-def**

配置 serializer max96717 的 i2c 默认上电的 7 位地址。

- **ser-init-sequence**

配置 serializer max96717 的 init sequence，一般是和摄像头硬件相关的配置。

2.2.5.2 Remote device sensor 的 DTS 配置

Remote sensor 的 DTS 配置和普通 sensor 类似，主要的变动是电源控制、reset 控制、clock 等，同时添加了地址映射以及 serializer 关联等。以 max96712_dphy3_cam0 为例：

```

max96712_dphy3_cam0: sc320at@31 {
    compatible = "maxim,smartsens,sc320at";
    reg = <0x31>;

    cam-i2c-addr-def = <0x30>;

    cam-remote-ser = <&max96712_dphy3_ser0>; // remote serializer

    poc-supply = <&max96712_dphy3_poc_regulator>;

    rockchip,camera-module-index = <0>;
    rockchip,camera-module-facing = "back";
    rockchip,camera-module-name = "default";
    rockchip,camera-module-lens-name = "default";

    /* port config start */
    port {
        max96712_dphy3_cam0_out: endpoint {
            /* remote endpoint: rkCIF_mipi_lvds_sditf */
            //remote-endpoint = <&mipi_lvds_sditf_in>;
            data-lanes = <1 2 3 4>;
        };
    };
    /* port config end */
};

```

- **sc320at@31**

配置 sensor sc320at 节点，i2c 地址使用的是映射后的 7 位新地址。

- **cam-i2c-addr-def**

配置 sensor sc320at 的 i2c 默认上电的7位地址。

- **cam-remote-ser**

配置摄像头关联的serializer phandle。

- **poc-supply**

配置远端摄像头的 PoC 上电的 regulator 输入。

- **port**

配置 sensor 的 port 属性。

注意：如果是 RAW 摄像头，remote-endpoint 链接到 ISP，用于曝光、增益等的配置。

2.3 Maxim Camera SerDes 近端解串器驱动

近端解串器驱动主要完成以下几个功能：

- 实现 deserializer dummy 摄像头抽象。
- 实现 deserializer pattern 模式的输出。
- 管理远端摄像头上下电和数据流的开关。
- 管理远端摄像头的通道开关、数据聚合映射。
- 实现远端摄像头的热拔插检测和恢复机制。

2.3.1 Local Deserializer - maxim4c

maxim4c 驱动目前支持 max96712、max96722 等四条 Link 的解串器，需要使能宏配置：CONFIG_VIDEO_MAXIM_DES_MAXIM4C，这里对主要的驱动模块和 API 函数进行说明。

- **maxim4c_api.h**

整个 maxim4c 主要模块的 API 接口函数定义、头文件管理、模块功能宏配置等。

```
#define MAXIM4C_NAME      "maxim4c"

/* Maxim Deserializer Test Pattern */
#define MAXIM4C_TEST_PATTERN    0

/* Maxim Deserializer pwn on/off enable */
#define MAXIM4C_LOCAL_DES_ON_OFF_EN 0
```

MAXIM4C_NAME: 定义 maxim4c 驱动的名字。

MAXIM4C_TEST_PATTERN: deserializer 的 pattern 输出使能配置：0 – 关闭，1 – 使能。默认关闭，一般使用 pattern 测试硬件板子的 MIPI 通路的功能。

MAXIM4C_LOCAL_DES_ON_OFF_EN: deserializer 运行模式：0 – deserializer 关流不掉电，1 – deserializer 关流正常掉电。默认关闭，节省开关流的配置时间。

- **maxim4c_drv.c**

整个 maxim4c 的 I2C 设备驱动入口，实现 maxim4c 的 I2C 设备驱动的加载，远端 i2c-mux 设备的加载，芯片 ID 的检测，电源管理，拔插中断管理机制，DTS 数据解析总入口，模块硬件初始化总入口，虚拟摄像头的 v4l2 subdev 初始化等，主要的 API 函数如下所示。

- I2C 通用的读写函数

```
int maxim4c_i2c_write(struct i2c_client *client,
                      u16 reg_addr, u16 reg_len, u32 val_len, u32 reg_val)

int maxim4c_i2c_read(struct i2c_client *client,
                     u16 reg_addr, u16 reg_len, u32 val_len, u32 *reg_val)

int maxim4c_i2c_update(struct i2c_client *client,
                       u16 reg_addr, u16 reg_len, u32 val_len, u32 val_mask, u32 reg_val)
```

- I2C 设备寄存器读写操作

```
int maxim4c_i2c_write_reg(struct i2c_client *client,
                           u16 reg_addr, u8 reg_val)

int maxim4c_i2c_read_reg(struct i2c_client *client,
                           u16 reg_addr, u8 *reg_val)

int maxim4c_i2c_update_reg(struct i2c_client *client,
                            u16 reg_addr, u8 val_mask, u8 reg_val)
```

- I2C 序列的相关操作函数

```
int maxim4c_i2c_write_array(struct i2c_client *client,
                             const struct maxim4c_i2c_regval *regs)

int maxim4c_i2c_load_init_seq(struct device *dev,
                              struct device_node *node, struct maxim4c_i2c_init_seq *init_seq)

int maxim4c_i2c_run_init_seq(struct i2c_client *client,
                              struct maxim4c_i2c_init_seq *init_seq)
```

- I2C-Mux 的操作函数

```
int maxim4c_i2c_mux_disable(maxim4c_t *maxim4c)

int maxim4c_i2c_mux_enable(maxim4c_t *maxim4c, u8 def_mask)

int maxim4c_i2c_mux_init(maxim4c_t *maxim4c)

int maxim4c_i2c_mux_deinit(maxim4c_t *maxim4c)
```

- ***maxim4c_mipi_txphy.c***

deserializer 的 MIPI TXPHY 的操作接口，主要包含 TXPHY 模式和 CLOCK 的配置、TXPHY 的开关使能、DPLL 的配置、CSI 输出使能等，主要的 API 函数如下所示。

- TXPHY 初始化和 DTS 解析

```
void maxim4c_mipi_txphy_data_init(maxim4c_t *maxim4c)

int maxim4c_mipi_txphy_hw_init(maxim4c_t *maxim4c)

int maxim4c_mipi_txphy_parse_dt(maxim4c_t *maxim4c, struct device_node
*of_node)
```

- TXPHY 使能配置函数

```
int maxim4c_mipi_txphy_enable(maxim4c_t *maxim4c, bool enable)
```

- TXPHY DPLL 配置函数

```
int maxim4c_dphy_dpll_predef_set(maxim4c_t *maxim4c, s64 link_freq_hz)
```

- TXPHY 输出使能函数

```
int maxim4c_mipi_csi_output(maxim4c_t *maxim4c, bool enable)
```

- ***maxim4c_link.c***

deserializer 的 Link 管理操作接口，主要功能包含 Link 的速率、类型、功能的配置、Link 的 Lock 状态获取、Link 的使能配置等，主要的 API 函数如下所示。

- Link 获取 Lock 状态

```
u8 maxim4c_link_get_lock_state(maxim4c_t *maxim4c, u8 link_mask)
```

- Link 等待 Lock 状态

```
int maxim4c_link_wait_linklock(maxim4c_t *maxim4c, u8 link_mask)
```

- Link oneshot 复位

```
int maxim4c_link_oneshot_reset(maxim4c_t *maxim4c, u8 link_mask)
```

- Link 开关使能

```
int maxim4c_link_mask_enable(maxim4c_t *maxim4c, u8 link_mask, bool enable)
```

- Link 通道使能

```
int maxim4c_link_select_remote_enable(maxim4c_t *maxim4c, u8 link_mask)
```

- Link 通道选择

```
int maxim4c_link_select_remote_control(maxim4c_t *maxim4c, u8 link_mask)
```

- Link 初始化和 DTS 解析

```
int maxim4c_link_parse_dt(maxim4c_t *maxim4c, struct device_node *of_node)
```

```
void maxim4c_link_data_init(maxim4c_t *maxim4c)
```

```
int maxim4c_link_hw_init(maxim4c_t *maxim4c)
```

- **maxim4c_video_pipe.c**

deserializer 的 video pipe 管理操作接口，主要功能包含 video pipe 的功能映射配置、video pipe 的选择和使能等，主要的 API 函数如下所示：

- Video Pipe 初始化和 DTS 解析

```
int maxim4c_video_pipe_hw_init(maxim4c_t *maxim4c)
```

```
void maxim4c_video_pipe_data_init(maxim4c_t *maxim4c)
```

```
int maxim4c_video_pipe_parse_dt(maxim4c_t *maxim4c, struct device_node *of_node)
```

- Video Pipe 使能开关

```
int maxim4c_video_pipe_mask_enable(maxim4c_t *maxim4c, u8
video_pipe_mask, bool enable)

int maxim4c_video_pipe_linkid_enable(maxim4c_t *maxim4c, u8
link_id, bool enable)
```

- ***maxim4c_pattern.c***

deserializer 的 test pattern 功能接口，可以输出分辨率为 1920X1080 的不同帧率的 RGB888 数据，主要的API 函数如下所示：

- Pattern 的初始化配置函数

```
int maxim4c_pattern_hw_init(maxim4c_t *maxim4c)

int maxim4c_pattern_data_init(maxim4c_t *maxim4c)

int maxim4c_pattern_support_mode_init(maxim4c_t *maxim4c)
```

- Pattern 使能配置函数

```
int maxim4c_pattern_enable(maxim4c_t *maxim4c, bool enable)
```

- ***maxim4c_remote.c***

deserializer 的 remote 设备管理接口，通过 `v4l2_subdev_call` 接口管理远端设备的上下电、开关流等功能，主要的 API 函数如 表3.1-8 所示。

- Remote 上下电控制

```
int maxim4c_remote_devices_power(maxim4c_t *maxim4c, u8 link_mask,
int on)
```

- Remote 开关流控制

```
int maxim4c_remote_devices_s_stream(maxim4c_t *maxim4c, u8
link_mask, int enable)
```

2.3.2 Local Deserializer - maxim2c

maxim2c 驱动目前支持 max96716、max96718 等两条 Link 的解串器，需要使能宏配置：CONFIG_VIDEO_MAXIM_DES_MAXIM2C，这里对主要的驱动模块和 API 函数进行说明。

- ***maxim2c_api.h***

整个 maxim2c 主要模块的 API 接口函数定义、头文件管理、模块功能宏配置等。


```

#define MAXIM2C_NAME      "maxim2c"

/* Maxim Deserializer Test Pattern */
#define MAXIM2C_TEST_PATTERN    0

/* Maxim Deserializer pwn on/off enable */
#define MAXIM2C_LOCAL_DES_ON_OFF_EN 0

```

MAXIM2C_NAME: 定义 maxim2c 驱动的名字。

MAXIM2C_TEST_PATTERN: deserializer 的 pattern 输出使能配置：0 – 关闭，1 – 使能。默认关闭，一般使用 pattern 测试硬件板子的 MIPI 通路的功能。

MAXIM2C_LOCAL_DES_ON_OFF_EN: deserializer 运行模式：0 – deserializer 关流不掉电，1 – deserializer 关流正常掉电。默认关闭，节省开关流的配置时间。

- **maxim2c_drv.c**

整个 maxim2c 的 I2C 设备驱动入口，实现 maxim2c 的 I2C 设备驱动的加载，远端 i2c-mux 设备的加载，芯片 ID 的检测，电源管理，拔插中断管理机制，DTS 数据解析总入口，模块硬件初始化总入口，虚拟摄像头的 v4l2 subdev 初始化等，主要的 API 函数如下所示。

- I2C 通用的读写函数

```

int maxim2c_i2c_write(struct i2c_client *client,
    u16 reg_addr, u16 reg_len, u32 val_len, u32 reg_val)

int maxim2c_i2c_read(struct i2c_client *client,
    u16 reg_addr, u16 reg_len, u32 val_len, u32 *reg_val)

int maxim2c_i2c_update(struct i2c_client *client,
    u16 reg_addr, u16 reg_len, u32 val_len, u32 val_mask, u32
    reg_val)

```

- I2C 设备寄存器读写操作

```

int maxim2c_i2c_write_reg(struct i2c_client *client,
    u16 reg_addr, u8 reg_val)

int maxim2c_i2c_read_reg(struct i2c_client *client,
    u16 reg_addr, u8 *reg_val)

int maxim2c_i2c_update_reg(struct i2c_client *client,
    u16 reg_addr, u8 val_mask, u8 reg_val)

```

- I2C 序列的相关操作函数

```
int maxim2c_i2c_write_array(struct i2c_client *client,
                           const struct maxim2c_i2c_regval *regs)

int maxim2c_i2c_load_init_seq(struct device *dev,
                             struct device_node *node, struct maxim2c_i2c_init_seq
                             *init_seq)

int maxim4c_i2c_run_init_seq(struct i2c_client *client,
                             struct maxim4c_i2c_init_seq *init_seq)
```

- I2C-Mux 的操作函数

```
int maxim2c_i2c_mux_disable(maxim2c_t *maxim2c)

int maxim2c_i2c_mux_enable(maxim2c_t *maxim2c, u8 def_mask)

int maxim2c_i2c_mux_init(maxim2c_t *maxim2c)

int maxim2c_i2c_mux_deinit(maxim2c_t *maxim2c)
```

- ***maxim2c_mipi_txphy.c***

deserializer 的 MIPI TXPHY 的操作接口，主要包含 TXPHY 模式和 CLOCK 的配置、TXPHY 的开关使能、DPLL 的配置、CSI 输出使能等，主要的 API 函数如下所示。

- TXPHY 初始化和 DTS 解析

```
void maxim2c_mipi_txphy_data_init(maxim2c_t *maxim2c)

int maxim2c_mipi_txphy_hw_init(maxim2c_t *maxim2c)

int maxim2c_mipi_txphy_parse_dt(maxim2c_t *maxim2c, struct
device_node *of_node)
```

- TXPHY 使能配置函数

```
int maxim2c_mipi_txphy_enable(maxim2c_t *maxim2c, bool enable)
```

- TXPHY DPLL 配置函数

```
int maxim2c_dphy_dpll_predef_set(maxim2c_t *maxim2c, s64
link_freq_hz)
```

- TXPHY 输出使能函数

```
int maxim2c_mipi_csi_output(maxim2c_t *maxim2c, bool enable)
```

- **maxim2c_link.c**

deserializer 的 Link 管理操作接口，主要功能包含 Link 的速率、类型、功能的配置、Link 的 Lock 状态获取、Link 的使能配置等，主要的 API 函数如下所示。

- Link 获取 Lock 状态

```
u8 maxim2c_link_get_lock_state(maxim2c_t *maxim2c, u8 link_mask)
```

- Link 等待 Lock 状态

```
int maxim2c_link_wait_linklock(maxim2c_t *maxim2c, u8 link_mask)
```

- Link oneshot 复位

```
int maxim2c_link_oneshot_reset(maxim2c_t *maxim2c, u8 link_mask)
```

- Link 开关使能

```
int maxim2c_link_mask_enable(maxim2c_t *maxim2c, u8 link_mask, bool enable)
```

- Link 通道使能

```
int maxim2c_link_select_remote_enable(maxim2c_t *maxim2c, u8 link_mask)
```

- Link 通道选择

```
int maxim2c_link_select_remote_control(maxim2c_t *maxim2c, u8 link_mask)
```

- Link 初始化和 DTS 解析

```
int maxim2c_link_parse_dt(maxim2c_t *maxim2c, struct device_node *of_node)

void maxim2c_link_data_init(maxim2c_t *maxim2c)

int maxim2c_link_hw_init(maxim2c_t *maxim2c)
```

- **maxim2c_video_pipe.c**

deserializer 的 video pipe 管理操作接口，主要功能包含 video pipe 的功能映射配置、video pipe 的选择和使能等，主要的 API 函数如下所示：

- Video Pipe 初始化和 DTS 解析

```
int maxim2c_video_pipe_hw_init(maxim2c_t *maxim2c)

void maxim2c_video_pipe_data_init(maxim2c_t *maxim2c)

int maxim2c_video_pipe_parse_dt(maxim2c_t *maxim2c, struct
device_node *of_node)
```

- Video Pipe 使能开关

```
int maxim2c_video_pipe_mask_enable(maxim2c_t *maxim2c, u8
video_pipe_mask, bool enable)

int maxim2c_video_pipe_linkid_enable(maxim2c_t *maxim2c, u8
link_id, bool enable)
```

- ***maxim2c_pattern.c***

deserializer 的 test pattern 功能接口，可以输出分辨率为 1920X1080 的不同帧率的 RGB888 数据，主要的API 函数如下所示：

- Pattern 的初始化配置函数

```
int maxim2c_pattern_hw_init(maxim2c_t *maxim2c)

int maxim2c_pattern_data_init(maxim2c_t *maxim2c)

int maxim2c_pattern_support_mode_init(maxim2c_t *maxim2c)
```

- Pattern 使能配置函数

```
int maxim2c_pattern_enable(maxim2c_t *maxim2c, bool enable)
```

- ***maxim2c_remote.c***

deserializer 的 remote 设备管理接口，通过 `v4l2_subdev_call` 接口管理远端设备的上下电、开关流等功能，主要的 API 函数如表3.1-8所示。

- Remote 上下电控制

```
int maxim2c_remote_devices_power(maxim2c_t *maxim2c, u8 link_mask,
int on)
```

- Remote 开关流控制

```
int maxim2c_remote_devices_s_stream(maxim2c_t *maxim2c, u8
link_mask, int enable)
```

2.4 Maxim Camera SerDes 远端摄像头驱动

Maxim Camera SerDes V3 版本驱动将车载远端的摄像头模组使用 deserializer 的反向通道选择的特性并结合 I2C-MUX 技术，实现远端设备的通道管理，同时将远端的serializer驱动使用统一的API 接口实现远近端设备的灵活搭配。

2.4.1 Remote Serializer

Remote serializer 定义一个结构体 `struct maxim_remote_ser` 和操作 `struct maxim_remote_ser_ops` 将不同的 serializer 统一管理起来，serializer 驱动在 probe 时绑定到 Remote sensor 上，实现serializer 和 sensor 的设备关联组合。

Remote serializer 驱动的主要功能提供 I2C 地址映射、输出 Sensor 所需的 MCLK、Sensor 的 Reset 等控制信号。

- ***maxim_remote.h***

Remote serializer 一体化的功能接口定义，定义了数据结构体 `struct maxim_remote_ser` 和操作结构体 `struct maxim_remote_ser_ops` 来管理数据和操作。

struct maxim_remote_ser 结构体：

```
typedef struct maxim_remote_ser {
    struct i2c_client *client;

    struct mutex mutex;

    u8 ser_i2c_addr_def;
    u8 ser_i2c_addr_map;

    u8 cam_i2c_addr_def;
    u8 cam_i2c_addr_map;

    u32 ser_state;

    struct maxim_remote_init_seq ser_init_seq;
    const struct maxim_remote_ser_ops *ser_ops;
} maxim_remote_ser_t;
```

- ***ser_i2c_addr_def***
serializer I2C 上电默认的7位地址，由模组硬件配置。
- ***ser_i2c_addr_map***
serializer I2C 重映射7位地址，驱动根据 DTS 配置进行地址重映射。
- ***cam_i2c_addr_def***
camera I2C 上电默认的7位地址，由模组硬件配置。
- ***cam_i2c_addr_map***
camera I2C 重映射 7 位地址，驱动根据 DTS 配置进行地址重映射。
- ***ser_state***
serializer 初始化状态，0：未初始化， 1：初始化完成。
- ***ser_init_seq***
serializer I2C 配置序列，根据模组硬件情况实际配置，配置数据从 DTS 加载。
- ***ser_ops***
serializer ops 操作函数指针，用于存储 serializer 统一操作接口函数。

struct maxim_remote_ser_ops 结构体：

```
struct maxim_remote_ser_ops {
    int (*ser_module_init)(struct maxim_remote_ser *remote_ser);
    int (*ser_module_deinit)(struct maxim_remote_ser *remote_ser);
    int (*ser_pclk_detect)(struct maxim_remote_ser *remote_ser);
};
```

- ***ser_module_init***
serializer 模块初始化，主要完成 serializer ID 校验、I2C 地址重映射、I2C 配置序列的执行等功能。
- ***ser_module_deinit***
serializer 模块反初始化。
- ***ser_pclk_detect***
serializer PCLK 检测，通过检测 PCLK 状态，可以判断 serializer 是否正常接收到 camera 的数据。

Remote serializer 主要的 API 函数如下所示：

- Remote serializer 的 I2C 序列操作函数

```
static inline int maxim_remote_parse_init_seq(struct device *dev,
const u8 *seq_data, int data_len, struct maxim_remote_init_seq
*init_seq)

static inline int maxim_remote_load_init_seq(struct device *dev,
struct device_node *node, struct maxim_remote_init_seq *init_seq)
```

- Remote serializer 的 I2C 地址选择

```
static inline int
maxim_remote_ser_i2c_addr_select(maxim_remote_ser_t *remote_ser,
u32 i2c_id)
```

- Remote serializer 的绑定 sensor

```
static inline struct maxim_remote_ser
*maxim_remote_cam_bind_ser(struct device *cam_dev)
```

- Remote serializer 的 init 状态

```
static inline bool maxim_remote_ser_is_initied(maxim_remote_ser_t
*remote_ser)
```

- **max9295.c**

remote serializer max9295 的设备驱动，主要实现 serializer 的 ID 检测、I2C 地址映射、PCLK 检测、初始化和反初始化操作等，主要 API 函数如下所示：

- Serializer I2C 设备驱动的 probe 和 remove

```
static int max9295_probe(struct i2c_client *client,
const struct i2c_device_id *id)

static int max9295_remove(struct i2c_client *client)
```

- Serializer I2C 读写操作函数

```
static int max9295_i2c_write(struct i2c_client *client,
u16 reg_addr, u16 reg_len, u32 val_len, u32 reg_val)

static int max9295_i2c_read(struct i2c_client *client,
u16 reg_addr, u16 reg_len, u32 val_len, u32 *reg_val)

static int max9295_i2c_update(struct i2c_client *client,
```

```

    u16 reg_addr, u16 reg_len, u32 val_len, u32 val_mask, u32
    reg_val)

static int max9295_write_reg(struct i2c_client *client,
    u16 reg_addr, u8 reg_val)

static int max9295_read_reg(struct i2c_client *client,
    u16 reg_addr, u8 *reg_val)

static int max9295_update_reg(struct i2c_client *client,
    u16 reg_addr, u8 val_mask, u8 reg_val)

```

- Serializer I2C 序列操作函数

```

static int max9295_run_ser_init_seq(struct i2c_client *client,
    struct maxim_remote_init_seq *init_seq)

static int max9295_load_ser_init_seq(maxim_remote_ser_t *max9295)

```

- Serializer I2C 地址映射

```

static int max9295_i2c_addr_remap(maxim_remote_ser_t *max9295)

```

- Serializer DTS 解析

```

static int max9295_parse_dt(maxim_remote_ser_t *max9295)

```

- Serializer 初始化和反初始化

```

static int max9295_module_init(maxim_remote_ser_t *max9295)

static int max9295_module_deinit(maxim_remote_ser_t *max9295)

```

- Serializer PCLK检测

```

static int max9295_pclk_detect(maxim_remote_ser_t *max9295)

```

- Serializer ID检测

```

static int max9295_check_chipid(maxim_remote_ser_t *max9295)

```

- **max96715.c**

remote serializer max96715 的设备驱动，主要实现 serializer 的 ID 检测、I2C 地址映射、PCLK 检测、初始化和反初始化操作等，主要 API 函数如下所示：

- Serializer I2C 设备驱动的 probe 和 remove

```
static int max96715_probe(struct i2c_client *client,
                          const struct i2c_device_id *id)

static int max96715_remove(struct i2c_client *client)
```

- Serializer I2C 读写操作函数

```
static int max96715_i2c_write(struct i2c_client *client,
                              u16 reg_addr, u16 reg_len, u32 val_len, u32 reg_val)

static int max96715_i2c_read(struct i2c_client *client,
                              u16 reg_addr, u16 reg_len, u32 val_len, u32 *reg_val)

static int max96715_i2c_update(struct i2c_client *client,
                                u16 reg_addr, u16 reg_len, u32 val_len, u32 val_mask, u32
                                reg_val)

static int max96715_write_reg(struct i2c_client *client,
                              u16 reg_addr, u8 reg_val)

static int max96715_read_reg(struct i2c_client *client,
                              u16 reg_addr, u8 *reg_val)

static int max96715_update_reg(struct i2c_client *client,
                                u16 reg_addr, u8 val_mask, u8 reg_val)
```

- Serializer I2C 序列操作函数

```
static int max96715_run_ser_init_seq(struct i2c_client *client,
                                      struct maxim_remote_init_seq *init_seq)

static int max96715_load_ser_init_seq(maxim_remote_ser_t *max96715)
```

- Serializer I2C 地址映射

```
static int max96715_i2c_addr_remap(maxim_remote_ser_t *max96715)
```

- Serializer DTS 解析

```
static int max96715_parse_dt(maxim_remote_ser_t *max96715)
```

- Serializer 初始化和反初始化

```
static int max96715_module_init(maxim_remote_ser_t *max96715)

static int max96715_module_deinit(maxim_remote_ser_t *max96715)
```

- Serializer PCLK 检测

```
static int max96715_pclk_detect(maxim_remote_ser_t *max96715)
```

- Serializer ID 检测

```
static int max96715_check_chipid(maxim_remote_ser_t *max96715)
```

- ***max96717.c***

remote serializer max96717 的设备驱动，主要实现 serializer 的 ID 检测、I2C 地址映射、PCLK 检测、初始化和反初始化操作等，主要 API 函数如下所示：

- Serializer I2C 设备驱动的 probe 和 remove

```
static int max96717_probe(struct i2c_client *client,
                          const struct i2c_device_id *id)

static int max96717_remove(struct i2c_client *client)
```

- Serializer I2C 读写操作函数

```
static int max96717_i2c_write(struct i2c_client *client,  
    u16 reg_addr, u16 reg_len, u32 val_len, u32 reg_val)  
  
static int max96717_i2c_read(struct i2c_client *client,  
    u16 reg_addr, u16 reg_len, u32 val_len, u32 *reg_val)  
  
static int max96717_i2c_update(struct i2c_client *client,  
    u16 reg_addr, u16 reg_len, u32 val_len, u32 val_mask, u32  
reg_val)  
  
static int max96717_write_reg(struct i2c_client *client,  
    u16 reg_addr, u8 reg_val)  
  
static int max96717_read_reg(struct i2c_client *client,  
    u16 reg_addr, u8 *reg_val)  
  
static int max96717_update_reg(struct i2c_client *client,  
    u16 reg_addr, u8 val_mask, u8 req_val)
```

- Serializer I2C 序列操作函数

```
static int max96717_run_ser_init_seq(struct i2c_client *client,
                                     struct maxim_remote_init_seq *init_seq)

static int max96717_load_ser_init_seq(maxim_remote_ser_t *max96717)
```

- Serializer I2C 地址映射

```
static int max96717_i2c_addr_remap(maxim_remote_ser_t *max96717)
```

- Serializer DTS 解析

```
static int max96717_parse_dt(maxim_remote_ser_t *max96717)
```

- Serializer 初始化和反初始化

```
static int max96717_module_init(maxim_remote_ser_t *max96717)

static int max96717_module_deinit(maxim_remote_ser_t *max96717)
```

- Serializer PCLK 检测

```
static int max96717_pclk_detect(maxim_remote_ser_t *max96717)
```

- Serializer ID 检测

```
static int max96717_check_chipid(maxim_remote_ser_t *max96717)
```

2.4.2 Remote Sensor

Maxim Camera SerDes V3 版本驱动对远端的 sensor 进行了 v4l2 subdev抽象，同时在 sensor 驱动中添加了serializer 的控制。

对于 YUV 格式输出的摄像头，一般情况下无需关注具体的 sensor 寄存器配置，摄像头模组上电后自动加载并完成配置工作，目前驱动中支持的 ov231x、ox01f10、ox03j10、sc320at 都是 YUV 输出的摄像头模组，如果不明确摄像头模组使用的 sensor 型号，那么可以使用 dummy 驱动来抽象。

对于 RAW 格式输出的摄像头，则需要在 sensor 驱动中实现初始化配置。

2.4.3 RAW 摄像头驱动适配

Maxim Camera SerDes 驱动框架中远端摄像头驱动和普通摄像头有一定的差异，这里重点介绍一下。远端摄像头驱动中，提供了 `dummy.c` 的 dummy sensor 参考以及 `os04a10` 实际 sensor 的驱动参考。

```
drivers/media/i2c/maxim/remote/dummy.c
drivers/media/i2c/maxim/remote/os04a10.c
```

2.4.3.1 Kconfig 和 Makefile 添加配置

```
drivers/media/i2c/maxim/remote/Kconfig
```

Kconfig 添加 dummy sensor 的配置：

```
config VIDEO_MAXIM_CAM_DUMMY
    tristate "Maxim Remote Sensor du
    depends on VIDEO_MAXIM_SERDES
    help
        This driver supports the remot

        To compile this driver as a mo
        module will be called dummy.
```

Makefile 添加 dummy sensor 的配置：

```
maxim-dummy-objs := dummy.o

obj-$(CONFIG_VIDEO_MAXIM_CAM_DUMMY) += maxim-dummy.o
```

注意：这里为了 modudle ko 名字一致冲突，添加了前缀。

2.4.3.2 模块驱动名字不同

Maxim Camera SerDes 远端摄像头驱动 为了确保不和普通摄像头驱动冲突，特地将驱动 `name` 和 `compatible` 添加前缀 `maxim` 便以区分：

`drivers/media/i2c/maxim/remote/dummy.c` 中实例代码如下：

```
static const struct of_device_id sensor_of_match[] = {
    { .compatible = "maxim,dummy,sensor" },
    { /* sentinel */ },
};
MODULE_DEVICE_TABLE(of, sensor_of_match);
```

```
static struct i2c_driver sensor_i2c_driver = {
    .driver = {
        .name = "maxim-dummy",
        .pm = &sensor_pm_ops,
        .of_match_table = of_match_ptr(sensor_of_match),
    },
    .probe = &sensor_probe,
    .remove = &sensor_remove,
};

module_i2c_driver(sensor_i2c_driver);
```

drivers/media/i2c/maxim/remote/os04a10.c 中实例代码如下：

```
static const struct of_device_id os04a10_of_match[] = {
    { .compatible = "maxim,ovti,os04a10" },
    { /* sentinel */ },
};
MODULE_DEVICE_TABLE(of, os04a10_of_match);

static struct i2c_driver os04a10_i2c_driver = {
    .driver = {
        .name = "maxim-os04a10",
        .pm = &os04a10_pm_ops,
        .of_match_table = of_match_ptr(os04a10_of_match),
    },
    .probe = &os04a10_probe,
    .remove = &os04a10_remove,
};

module_i2c_driver(os04a10_i2c_driver);
```

2.4.3.3 MCLK 提供方式不同

SerDes 远端摄像头驱动 MCLK 可以由晶振提供，也可以由Serializer提供，如果是Serializer提供，那么需要Serializer实现相应的配置输出。

普通摄像头一般由SoC提供，所以普通摄像头驱动适配到 SerDes 摄像头驱动时会去除MCLK相关部分的代码：包括时钟的申请、使能和频率配置等。

2.4.3.4 电源提供方式不同

SerDes 远端摄像头采用PoC方式供电，在sensor驱动结构体 `struct sensor` 中添加 `poc_regulator` 电源管理：

```
struct regulator    *poc_regulator; /* PoC */
```

函数 `sensor_probe` 相关的代码参考如下：

```
// poc regulator
sensor->poc_regulator = devm_regulator_get(dev, "poc");
if (IS_ERR(sensor->poc_regulator)) {
    if (PTR_ERR(sensor->poc_regulator) != -EPROBE_DEFER)
        dev_err(dev, "Unable to get PoC regulator (%ld)\n",
            PTR_ERR(sensor->poc_regulator));
    else
        dev_err(dev, "Get PoC regulator deferred\n");

    ret = PTR_ERR(sensor->poc_regulator);

    return ret;
}
```

普通摄像头一般由SoC提供，驱动中的 `xxx_supply_names` 结构体和 `xxx_configure_regulators` 函数来配置相关的电源，所以在适配到 SerDes 摄像头驱动时，这部分代码也要同步修改。主要涉及的函数参考：`__sensor_power_on` 和 `__sensor_power_off`

2.4.3.5 控制时序 不同

SerDes 远端摄像头 根据实际的硬件设计将sensor的reset、power和pwrn等控制连接到 Serializer，所以需要Serializer实现相关的控制。

普通摄像头一般由SoC提供，所以在适配到 SerDes 摄像头驱动时，要把相关的gpio申请和控制去掉，主要涉及的函数参考：`sensor_probe`、`__sensor_power_on` 和 `__sensor_power_off` 等。

2.4.3.6 chip id 检测位置不同

普通摄像头在probe阶段就识别 sensor 的 chip id，SerDes 框架中 远端摄像头probe的时候，serdes 的link都尚未准备好，i2c无法正常通讯，所以将chip id的检测放在 `__sensor_start_stream` 的时检测。

2.4.3.7 probe 添加 serializer 绑定关联

SerDes 远端摄像头的 serializer 和 sensor 组合很多，驱动框架通过i2c-mux来实现这种灵活组合，所以在sensor驱动probe的时候需要通过phandle来绑定：

```

/* remote serializer bind */
serializer = sensor_get_serializer_by_phandle(dev);
if (serializer != NULL) {
    dev_info(dev, "serializer bind success\n");

    serializer->cam_i2c_addr_def = sensor->cam_i2c_addr_def;
    serializer->cam_i2c_addr_map = sensor->cam_i2c_addr_map;

    sensor->serializer = serializer;
} else {
    dev_err(dev, "serializer bind fail\n");

    sensor->serializer = NULL;
}

```

2.4.3.8 开关流接口适配

Maxim Camera SerDes 驱动框架中将不同类型的 serializer 进行统一抽象，并提供了操作接口，Sensor驱动在probe时，通过 phandle 绑定 serializer，获取相应的操作接口，开流时首先执行 serializer 初始化操作，再执行sensor相关的配置序列并执行出流，最后再检测pclk来确保摄像头数据被 serializer 正常接收。

开流函数 `__sensor_start_stream` 实现如下：

```

static int __sensor_start_stream(struct sensor *sensor)
{
    maxim_remote_ser_t *serializer = sensor->serializer;
    struct i2c_client *client = sensor->client;
    struct device *dev = &client->dev;
    int ret = 0;

    if (serializer == NULL) {
        dev_err(dev, "%s: serializer error\n", __func__);
        return -EINVAL;
    }

    if (serializer->ser_ops == NULL) {
        dev_err(dev, "%s: serializer ser_ops error\n", __func__);
        return -EINVAL;
    }

    ret = serializer->ser_ops->ser_module_init(serializer);
    if (ret) {
        dev_err(dev, "%s: serializer module_init error\n", __func__);
        return ret;
    }
}

```

```

ret = sensor_check_chip_id(sensor);
if (ret) {
    dev_err(dev, "%s: sensor check chip id error\n", __func__);
    return ret;
}

ret = sensor_i2c_write_array(client, sensor->cur_mode->reg_list);
if (ret) {
    dev_err(dev, "%s: sensor i2c write array error\n", __func__);
    return ret;
}

/* In case these controls are set before streaming */
ret = __v4l2_ctrl_handler_setup(&sensor->ctrl_handler);
if (ret)
    return ret;

/* streaming control register */
#if 0 /* TODO */
ret = sensor_i2c_write_reg(client,
    SENSOR_REG_CTRL_MODE,
    SENSOR_REG_VALUE_08BIT,
    SENSOR_MODE_STREAMING);
if (ret) {
    dev_err(dev, "%s: sensor start stream error\n", __func__);
    return ret;
}
#else
dev_info(dev, "%s: %d: TODO\n", __func__, __LINE__);
#endif

/* serializer pclk detect */
ret = serializer->ser_ops->ser_pclk_detect(serializer);
if (ret) {
    dev_err(dev, "%s: serializer pclk_detect error\n", __func__);
    return ret;
}

return 0;
}

```

关流函数 `__sensor_stop_stream` 实现如下：

```

static int __sensor_stop_stream(struct sensor *sensor)
{
    maxim_remote_ser_t *serializer = sensor->serializer;
    struct i2c_client *client = sensor->client;

```



```

    struct device *dev = &client->dev;
    int ret = 0;

    /* streaming control register */
    #if 0 /* TODO */
        ret = sensor_i2c_write_reg(client,
                                    SENSOR_REG_CTRL_MODE,
                                    SENSOR_REG_VALUE_08BIT,
                                    SENSOR_MODE_SW_STANDBY);
        if (ret) {
            dev_err(dev, "%s: sensor stop stream error\n", __func__);
            return ret;
        }
    #else
        dev_info(dev, "%s: %d: TODO\n", __func__, __LINE__);
    #endif

    if (serializer == NULL) {
        dev_err(dev, "%s: serializer error\n", __func__);
        return -EINVAL;
    }

    if (serializer->ser_ops == NULL) {
        dev_err(dev, "%s: serializer ser_ops error\n", __func__);
        return -EINVAL;
    }

    ret = serializer->ser_ops->ser_module_deinit(serializer);
    if (ret) {
        dev_err(dev, "%s: serializer module_deinit error\n", __func__);
        return ret;
    }

    return 0;
}

```

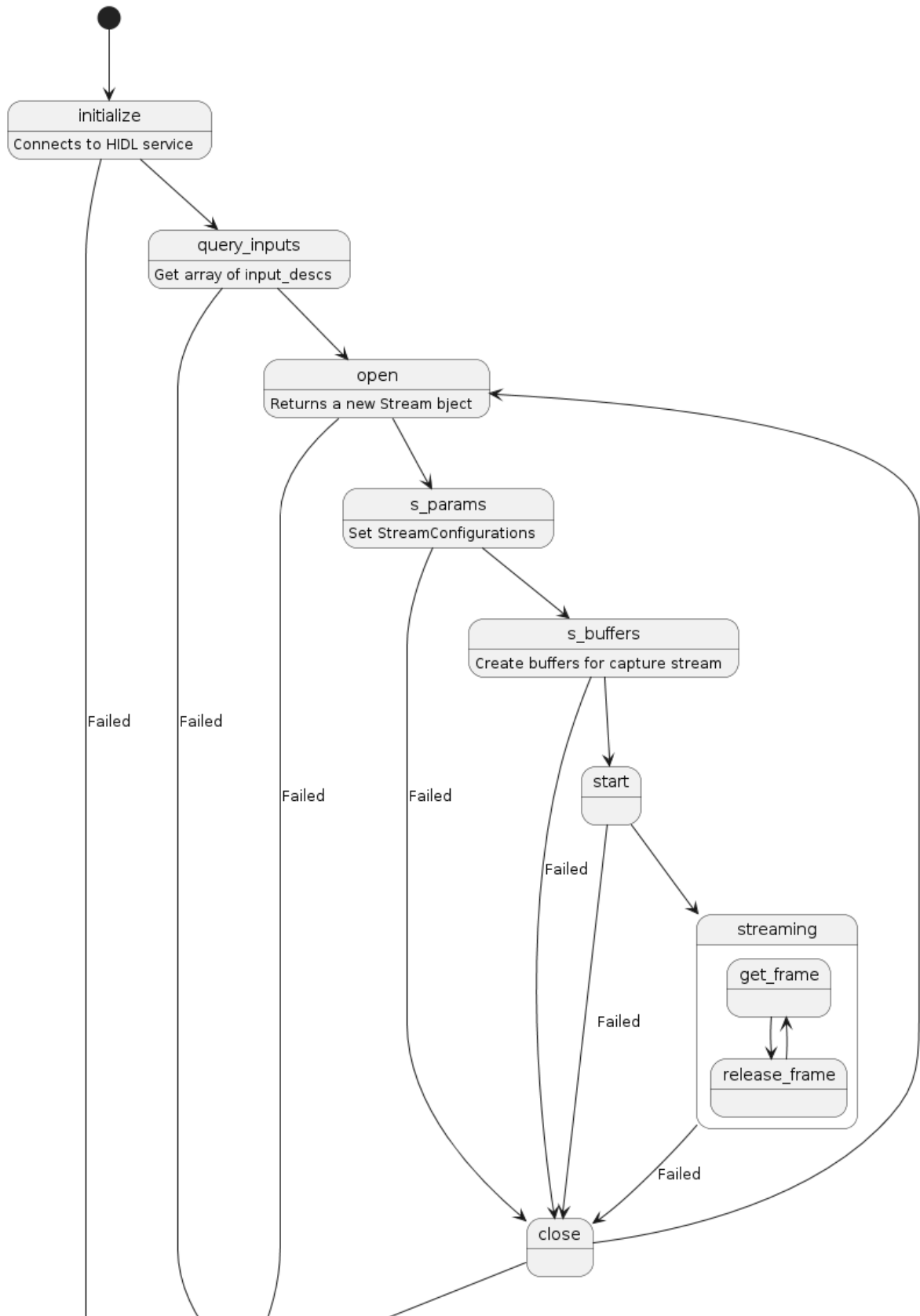
3. RVCAM 中间件

3.1 RVCAM 介绍

RVCAM 是 Rockchip Vehicle Camera 的缩写，是一款专为车载摄像头系统设计的软件框架，旨在简化摄像头数据的处理、管理和应用开发。它提供了一套统一的接口和工具，使开发者能够快速地在 AAOS、Linux 等系统构建高效、可靠的车载摄像头应用。

3.2 RVCAM API 说明

RVCAM API 的调用流程如下图。



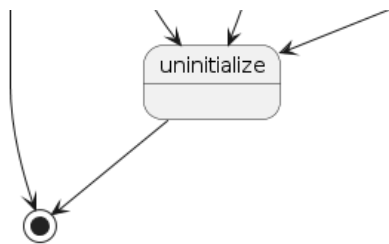


图 RVCAM API 状态机

3.2.1 初始化与终止

- **rvcam_initialize**

```
int rvcam_initialize(rvcam_init_t* p_init_params);
```

功能描述：使用提供的初始化参数 (`rvcam_init_t`) 初始化上下文并连接到 RVCAM 服务器。

参数：

`p_init_params`：指向初始化参数结构的指针。

返回值：成功返回 0，否则返回非零值。

参见： `rvcam_uninitialize`

- **rvcam_uninitialize**

```
int rvcam_uninitialize(void);
```

功能描述：断开与 RVCAM 服务器的连接并释放上下文。

返回值：成功返回 0，否则返回非零值。

参见： `rvcam_initialize`

3.2.2 查询输入信息

- **rvcam_query_inputs**

```
int rvcam_query_inputs(rvcam_input_t* p_inputs,  
                      unsigned int size,  
                      unsigned int* ret_size);
```

功能描述：从 RVCAM 获取所有输入信息。此函数需调用两次：

首先，以 `p_inputs = NULL` 调用，获取所需数组大小 (`ret_size`)。
根据获取的大小为 `p_inputs` 分配内存，再次调用以获取输入信息。
参数：

`p_inputs`：RVCAM 将填充输入信息的内存数组指针，首次调用时可为 NULL。

`size`： `p_inputs` 数组的大小。

`ret_size`：用于存储 `p_inputs` 数组实际填充大小的指针。

返回值：成功返回 0，否则返回非零值。

参见： `rvacam_input_t`

3.2.3 查询诊断信息

- `rvacam_query_diagnostics`

```
int rvacam_query_diagnostics(void* p_diag_info,  
                             unsigned int diag_size);
```

功能描述：目前尚未实现，旨在从 RVCAM 获取诊断信息。

参数：

`p_diag_info`：诊断信息将被填充的内存指针。

`diag_size`： `p_diag_info` 内存的大小。

返回值：成功返回 0，否则返回非零值。

参见： `rvacam_query_inputs`

3.2.4 管理虚拟摄像头流

- `rvacam_open`

```
rvacam_hndl_t rvacam_open(int id);
```

功能描述：使用指定的虚拟摄像头 ID 打开摄像头流，返回一个句柄用于后续操作。

参数：

`id`：待打开虚拟摄像头的 ID。

返回值：返回访问已打开虚拟摄像头的句柄 (`rvacam_hndl_t`)，若打开操作失败则返回 NULL。

参见： `rvacam_hndl_t`, `rvacam_close`

- `rvacam_close`

```
int rvcam_close(rvcam_hdl_t hndl);
```

功能描述： 关闭与给定句柄关联的摄像头流。

参数：

hndl： 待关闭的虚拟摄像头句柄。

返回值： 成功返回 0，否则返回非零值。

参见： `rvcam_hdl_t`, `rvcam_open`

- `rvcam_start`

```
int rvcam_start(rvcam_hdl_t hndl);
```

功能描述： 开始与句柄关联的摄像头流。

参数：

hndl： 待开始的虚拟摄像头句柄。

返回值： 成功返回 0，否则返回非零值。

参见： `rvcam_hdl_t`, `rvcam_stop`

- `rvcam_stop`

```
int rvcam_stop(rvcam_hdl_t hndl);
```

功能描述： 停止与句柄关联的摄像头流。

参数：

hndl： 待 停止的虚拟摄像头句柄。

返回值： 成功返回 0，否则返回非零值。

参见： `rvcam_hdl_t`, `rvcam_start`

- `rvcam_pause`

```
int rvcam_pause(rvcam_hdl_t hndl);
```

功能描述： 暂停句柄关联的摄像头流。

参数：

hndl： 待暂停的虚拟摄像头句柄。

返回值： 成功返回 0，否则返回非零值。

参见： `rvcam_hdl_t`, `rvcam_resume`

- **rvcam_resume**

```
int rvcam_resume(rvcam_hndl_t hndl);
```

功能描述： 恢复被暂停的与句柄关联的摄像头流。

参数：

hndl：待恢复的虚拟摄像头句柄。

返回值： 成功返回 0，否则返回非零值。

参见： **rvcam_hndl_t**, **rvcam_pause**

3.2.5 参数及配置管理

- **rvcam_g_param**

```
int rvcam_g_param(rvcam_hndl_t hndl,  
                  rvcam_param_t param,  
                  rvcam_param_value_t* p_value);
```

功能描述： 为给定虚拟摄像头获取指定参数的值。

参数：

hndl：虚拟摄像头句柄。

param：参数枚举索引 (**rvcam_param_t**)。

p_value：用于存储检索到的参数值 (**rvcam_param_value_t**) 的指针。

返回值： 成功返回 0，否则返回非零值。

参见： **rvcam_hndl_t**, **rvcam_param_t**, **rvcam_param_value_t**, **rvcam_s_param**

- **rvcam_s_param**

```
int rvcam_s_param(rvcam_hndl_t hndl,  
                  rvcam_param_t param,  
                  const rvcam_param_value_t* p_value);
```

功能描述： 为给定虚拟摄像头设置指定参数的值。

参数：

hndl：虚拟摄像头句柄。

param：参数枚举索引 (**rvcam_param_t**)。

p_value：指向待设置参数值 (**rvcam_param_value_t**) 的指针。

3.2.6 BUFFER 队列管理

- **rvcam_s_buffers**

```
int rvcam_s_buffers(rvcam_hndl_t hndl,
                    rvcam_buffers_t* p_buffers);
```

功能描述：为给定虚拟摄像头传入外部缓冲区队列。

参数：

hndl：虚拟摄像头句柄。

p_buffers：待传入的外部缓冲区列表(**rvcam_buffers_t**)。

返回值：成功返回 0，否则返回非零值。

参见：**rvcam_hndl_t**, **rvcam_buffers_t**

- **rvcam_s_input_buffers**

```
int rvcam_s_input_buffers(rvcam_hndl_t hndl,
                          rvcam_buffers_t* p_buffers);
```

功能描述：为给定虚拟摄像头传入摄像头采集缓冲区队列。

参数：

hndl：虚拟摄像头句柄。

p_buffers：待传入的缓冲区列表(**rvcam_buffers_t**)。

返回值：成功返回 0，否则返回非零值。

参见：**rvcam_hndl_t**, **rvcam_buffers_t**

- **rvcam_get_frame**

```
int rvcam_get_frame(rvcam_hndl_t hndl,
                    rvcam_frame_info_t* p_frame_info,
                    uint64_t timeout,
                    unsigned int flags);
```

功能描述：从虚拟摄像头已经采集处理完成缓冲区队列获取一帧信息，获取后该帧标记为被消费。

参数：

hndl：虚拟摄像头句柄。

p_frame_info：输入输出参数，待填充的帧信息结构体指针(**rvcam_frame_info_t**)。

timeout：获取超时，单位毫秒

flags：帧标记，暂无使用

返回值：成功返回 0，否则返回非零值。

参见： `rvcam_hndl_t`, `rvcam_frame_info_t`

- `rvcam_release_frame`

```
int rvcam_release_frame(rvcam_hndl_t hndl,
                        uint64_t idx);
```

功能描述：将 `idx` 对应的 buffer 重新加入回给虚拟摄像头缓冲区队列。

参数：

`hndl`：虚拟摄像头句柄。

`idx`：待释放的 buffer cookies

返回值：成功返回 0，否则返回非零值。

参见： `rvcam_hndl_t`, `rvcam_frame_info_t`

3.3 RVCAM 配置文件说明

RVCAM 的配置文件位于

`hardware/rockchip/rvcam/components/config/xml/rvcam_config.xml`，产品安装目录 `/vendor/etc/rvcam_config.xml`。允许用户在产品板级的 `mk` 里通过 `PRODUCT_COPY_FILES` 进行覆盖生效。

配置文件包含以下内容：

- `CameraConfig`：配置文件根节点，包含 `version` 属性，表示配置文件的版本号。
- `board`：相应的产品内核 dts 配置的 model 字符串，用于同一个配置文件兼容不同硬件。可以通过命令 `cat /proc/device-tree/model` 在设备上查看。
- `camera`：用于配置对应摄像头的信息，其中 `entity`、`subDev`、`mbusCode` 通过 `mediactl` 命令可以在设备上获取到，目前这些信息暂时未使用。
- `channels`：用于配置不同通道接的摄像头型号，数据流采集节点，数据流格式、分辨率等信息。

示例文件如下：

```
<?xml version="1.0" encoding="UTF-8"?>
<CameraConfig version="0x100">
  <board name="Rockchip RK3588 VEHICLE EVB V22 Board">
    <cameras>
      <camera id="0">
        <entity>m00_b_max96712 6-0029</entity>
        <subDev>/dev/v4l-subdev5</subDev>
```



```

        <mbusCode>UYVY2X8/1920x1440</mbusCode>
        <channels>
            <channel id="0" dev="/dev/video11">
                <cameraModel>TXD</cameraModel>
                <stream id="0" format="YUYV" width="1920" height="1440"
bufCnt="4"/>
            </channel>
            <channel id="1" dev="/dev/video12">
                <cameraModel>TXD</cameraModel>
                <stream id="0" format="YUYV" width="1920" height="1440"
bufCnt="4"/>
            </channel>
            <!-- ... -->
        </channels>
    </camera>
    <!-- ... -->
</CameraConfig>

```

同时，配置文件中包含需要配置向上层映射的虚拟摄像头节点 ID，如无映射，上层访问不到任何摄像头。

- `inputMapping`：摄像头映射根节点，包含多个映射节点
- `inputMap`：一组摄像头数据流映射，`camId` 属性为映射的虚拟摄像头 ID
- `inputSrc`：映射的摄像头来源，包含 `cameraId`、`channelId`、`streamId`，对应于上述摄像头配置。

```

    <!-- ... -->
    <inputMapping>
        <inputMap camId="0">
            <inputSrc cameraId="0" channelId="0" streamId="0"/>
        </inputMap>
        <inputMap camId="1">
            <inputSrc cameraId="0" channelId="1" streamId="0"/>
        </inputMap>
        <inputMap camId="2">
            <inputSrc cameraId="0" channelId="2" streamId="0"/>
        </inputMap>
        <inputMap camId="3">
            <inputSrc cameraId="0" channelId="3" streamId="0"/>
        </inputMap>
        <inputMap camId="4">
            <inputSrc cameraId="0" channelId="0" streamId="0"/>
        </inputMap>
    </inputMapping>
    <!-- ... -->

```

3.4 RVCAM 插件扩展

TBD

3.5 RVCAM 性能指标说明

3.5.1 CPU 使用率

以 RK3576M Android 14 平台为例，设置 CPU 定频 1GHz，DDR 定最高频率：

```
echo performance > /sys/devices/platform/dmc/devfreq/dmc/governor
echo userspace > /sys/devices/system/cpu/cpufreq/policy0/scaling_governor
echo 1008000 > /sys/devices/system/cpu/cpufreq/policy0/scaling_setspeed
```

打开 Camera360 APK AVM 界面，四路摄像头的分辨率为 1920x1080，帧率30fps，使用 `top` 命令查看 CPU 使用率，`vendor.rockchip.automotive.camera.rvcam@1.0-service` 服务平均 CPU 使用率在 10% 左右。

3.6 RVCAM 调试功能说明

debug helper 调试组件用于分析虚拟和物理摄像头状态，包括摄像头开关状态、帧率、是否丢帧等信息，相关数据保存在板端 `data/vendor/camera/rvcam_debug.log` 文件中，目前只支持 Android 系统。

以下 property 支持打开 debug helper 功能，并设置写入文件的时间间隔

```
setprop persist.vendor.rockchip.rvcam.dbgfile.on true

setprop persist.vendor.rockchip.rvcam.dbgfile.interval 200 // 不设置的话默认值为 200
```

3.7 RVCAM 示例 RVC 单摄应用

RVC 单摄应用位于 `/examples/rvc` 路径下，编译生成 `rvcam_rvc_la_u` 和 `rvcam_rvc_la_s` 两个示例，分别对应 Android 14 和 Android 12 两个系统版本。运行时需要先停止运行 `evs_app` 进程，否则会报错。

```
stop evs_app  
  
./rvcam_rvc_la_u
```

3.8 RVCAM 示例 AVM + EGL 应用

TBD

4. EVS

EVS（Exterior View System）是安卓系统上为车载（Automotive）设计的一套Camera框架，Android 的汽车 HIDL 硬件抽象层 (HAL) 可用于在 Android 启动过程的初期拍摄并显示图像，且可在系统的整个生命周期内持续运行。EVS通常用于在搭载基于 Android 的车载信息娱乐 (IVI) 系统的汽车上为后视摄像头和环绕视图显示提供支持。

4.1 EVS 模块划分与目录介绍

EVS的具体系统框架介绍可以参考官方官方文档，这里借鉴官方文档上的系统框架图。

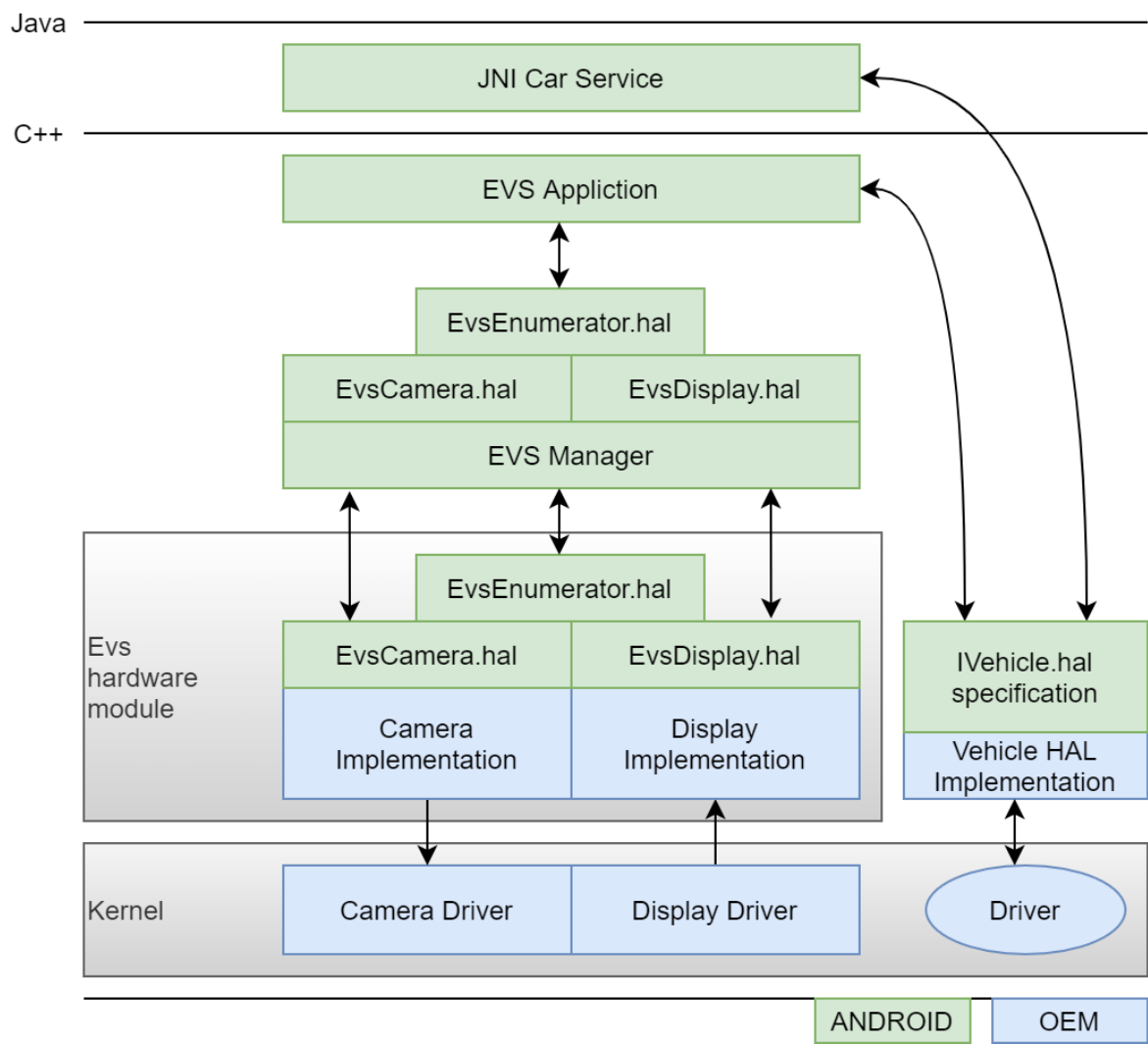


图 EVS 系统框架图

由上面的框架图上看，EVS框架主要包含以下几部分：EVS APP, EVS Manager, EVS HAL。下面展开介绍一下框架的主要部分。

4.1.1 Native 层目录介绍

4.1.1.1 EVS APP

EVS APP 源码位于SDK/packages/services/Car/cpp/evs/apps/default/，其函数入口位于evs_app.cpp

EVS APP启动时主要做了以下几个动作：

- 1) 解析输入参数实现差异化启动，可以在运行后--help了解具体参数
- 2) 初始化配置参数，通过解析配置文件里的具体内容获取硬件信息，应用启动时会优先解析override的配置参数

```
/vendor/etc/automotive/evs/config_override.json
```

该配置文件一般由产商来配置，如果override目录参数解析错误，才会去解析目录

```
/vendor/etc/automotive/evs/config.json
```

- 3) 获取底层Evs Display操作对象，Display将指定在多个屏幕之间的具体显示位置
- 4) 在需要使用Vehicle HAL的前提下，连接Vehicle HAL，订阅车辆挡位信息与转向灯信息。当数据变化时通过listener回调处理。
- 5) 开启EvsStateControl状态更新线程，在该线程内根据外部输入调整EVS APP的运行状态以及实现具体的图像绘制

4.1.1.2 EVS Manager

Android 12中EVS Manager的源码位于SDK/packages/services/Car/cpp/evs/manager/目录，这里存在两个版本，即1.0与1.1，对应EVS HAL的1.0和1.1版本。EVS Manager 1.1版本是向下兼容的，也就是说EVS HAL 1.0版本，EVS Manager 1.1版本也可以适配。

Android 14中EVS Manager的源码位于SDK/packages/services/Car/cpp/evs/manager/aidl，其向下兼容的EVS HAL aidl版本。

EVS Manager作为中间嵌套的一层，为APP提供接入EVS HAL的接口，同时实现对两个重要抽象对象（EVS Camera与EVS Display）的集中管理，其本质是对EVS HAL接口的一层封装，在此基础上实现了一些其他功能。比如数据统计，诊断相关以及虚拟Camera设备等

4.1.1.3 EVS HAL

EVS HAL作为隔离用户态程序（EVS APP）与底层驱动（Camera Driver）的HAL实现，是OEM厂商最为关注的部分。这部分也是原生EVS架构中最为复杂的部分。其源码路径位于/hardware/interfaces/automotive/evs目录。

其包含了1.0/1.1两个版本，Android 13及更高版本还实现了aidl版本；其中1.0版本有以下接口：

IEvsCamera.hal：该对象表示单个相机，是用于拍摄图像的主接口。

IEvsCameraStream.hal：客户端实现此接口以接收异步传送的视频帧。

IEvsDisplay.hal：该对象可表示 EVS 显示，控制显示状态，并处理图像的实际呈现效果。

IEvsEnumerator.hal：该对象负责枚举系统中可用的 EVS 硬件（一个或多个相机和单个显示设备）。

types.hal：该对象包含了EVS HAL的结构体信息。

1.1版本更新了IEvsUltrasonicsArray.hal和IEvsUltrasonicsArrayStream.hal，目前并未看到相关实现。

关于原生EVS HAL的实现，我们可以参考对于文件夹下的default目录，但这部分实现都是脱离于底层驱动的，不可以直接使用。为了便于理解，谷歌官方还实现了一个Sample，实现将EVS HAL于内核v4l2驱动相结合，用作具体实现上的一个参考，这里参考的代码来自于该Sample。

该Sample的源码路径于/packages/services/Car/cpp/evs/sampleDriver。Android 12实现的是hidl框架的示例，Android 14上更新了aidl框架的示例。程序入口位于service.cpp中的main函数，在main函数执行的过程中，我们可以看到以下几个环节：

- 1) 获取系统Display
- 2) 使能对Uevent事件的处理线程，该线程内将会处理设备添加，移除等
- 3) 注册为service，加入binder线程池

EVS HAL开始运行后，当上层应用调用到相应的接口，会由EVS Manager传递到EVS HAL，从而进入相应的接口执行。

4.1.2 JAVA层目录介绍

上面详细介绍了EVS的框架，其除了在Native层拥有三个服务（EVS HAL,EVS Manager,EVS APP）之外，在CarService中还提供了一个CarEvsManager与CarEvsService。CarService是Android Automotive所提供的核心服务，众多的Automotive特性都依赖于该服务。其源码位置位于SDK/packages/services/Car/service/

JNI CarEvsService 源码目录位于SDK/packages/services/Car/service/jni

CarEvsService通过JNI接口与Java层进行通信，实现了与汽车硬件和功能的交互。CarEvsService向下会链接EVS Manager，调用EVS Manager接口获取数据流。

4.1.3 Test APK

EVS Preview Sample

EVS Preview Sample源码位于SDK/packages/services/Car/tests/CarEvsCameraPreviewApp/目录，其主要实现了单路摄像头数据预览功能，其通路如下图所示：

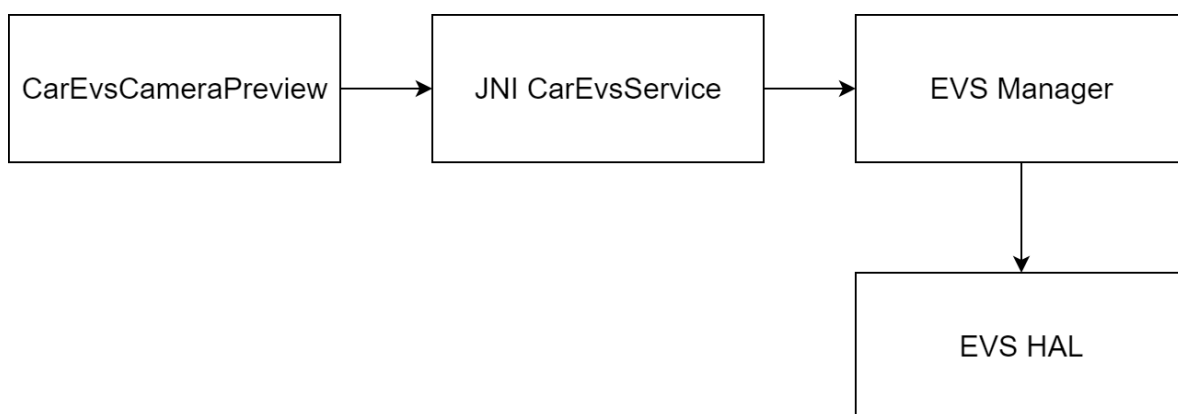


图 EVS Preview Sample数据流程图

4.2 EVS 和 VHAL (Vehicle HAL) 状态关系

车载硬件抽象层 (Vehicle HAL) 是与其他汽车控制器沟通的桥梁，其对下会接入 MCU/UART/ETHERNET等实现外部通信，对上接入CarService，服务于Java Framework，当然也可以接入Native Framework，比如EVS APP就可以接入VHAL来控制EVS APP的运行状态。下面分别从这两个层面介绍一下EVS与VHAL的状态关系

4.2.1 Native

EVS APP在主函数入口有实现接入VHAL的选项，其使用VHAL主要是为了监测外部输入来控制EVS APP运行状态，目前只监测了档位控制 (VehicleProperty::GEAR_SELECTION) 以及转向灯控制 (VehicleProperty::TURN_SIGNAL_STATE) 这两个属性。其运行状态关系如下图所示：

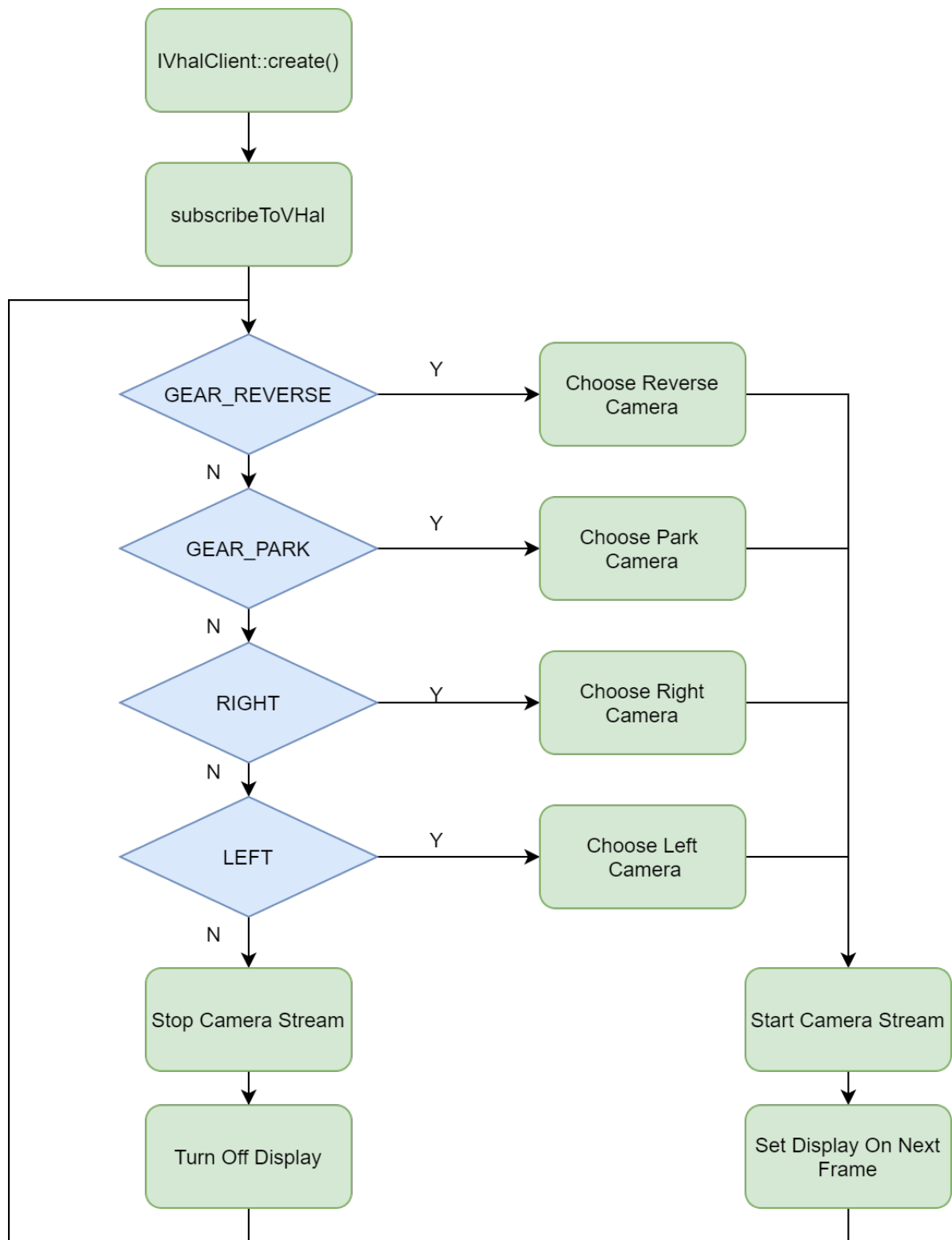


图 EVS APP与VHAL状态关系流程图

4.2.2 Java

当VHAL接入Java Framework时，其工作状态与接入EVS APP完全不同，一个是EVS APP中接入VHAL只用到了档位控制和转向灯控制两个属性，相对比较简单；另一个是EVS APP接入VHAL后监测输入变化时是EVS APP内部状态改变，对VHAL来说并没有回调相应的结果。而VHAL接入Java Framework时，其工作状态大致如下图所示：上层的应用通过 CarAPI提供的接口进行设置，最终通过车辆总线将命令发送给对应的电子控制单元(ElectronicControlUnit,ECU)，ECU 返回结果给 CarService,并通过回调通知相关应用。VHAL的控制属性定义在 SDK/hardware/interfaces/automotive/vehicle/2.0/types.hal文件中。

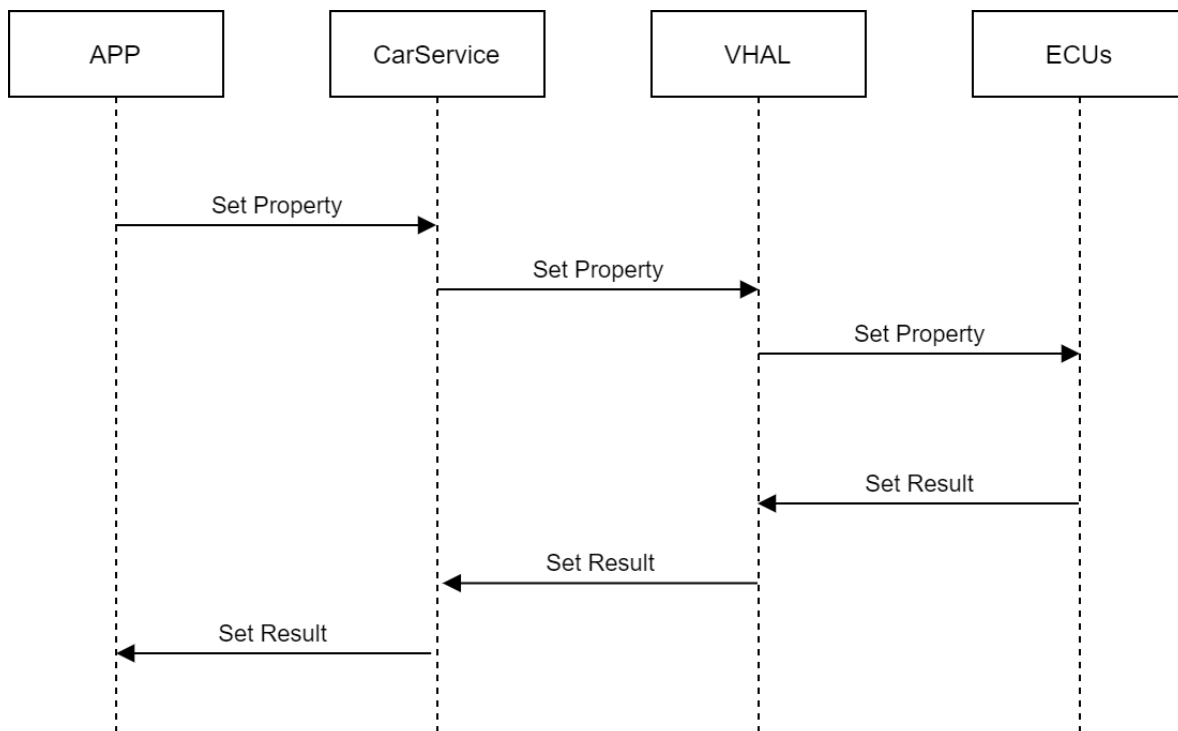


图 Java Framework与VHAL状态关系流程图

4.3 框架流程图

4.3.1 Camera 数据框架流程图

Camera数据流程图如下图所示：EVS APP在主函数入口开启EvsStateControl状态更新线程，根据外部输入的变化来选择相应的Render进行图像绘制，在Render中通过createVideoTex函数创建对应的Camera Context打开对应的Camera硬件，然后EVS APP在refresh函数内处理Camera数据，涉及到的接口包括：deliverFrame，newFrameAvailable，getNewFrame与doneWithFrame。其中deliverFrame与doneWithFrame都属于HIDL接口，其具体流程如下：当EVS HAL获取到新的帧时，会通过deliverFrame接口将数据上报给EVS APP，EVS APP会将其存储至一个双Buffer中（一个用于存储新的Camera数据，一个用于存储当前正在使用中的Camera数据，两者循环），在EVS APP绘制时，先通过newFrameAvailable接口判断当前是否有新的数据到来，如存在新的数据，则通过getNewFrame来获取新的camera数据，在使用OpenGL ES接口进行绘制渲染后，再通过 doneWithFrame 将buffer归还给底层。当线程结束时，通过调用

deactive函数Stop数据流，只有当进程完全析构时，才会Close Camera。

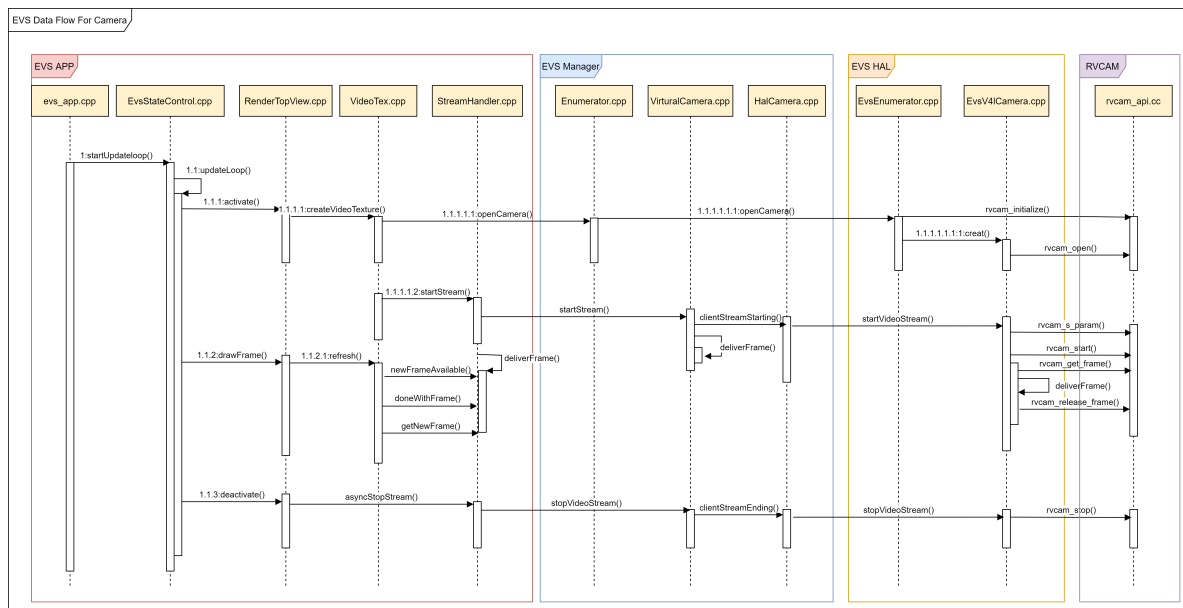


图 EVS Camera数据框架流程图

4.3.2 显示数据框架流程图

显示数据流程图如下图所示：EVS APP通过主函数入口startUpdateLoop函数开启 EvsStateControl状态更新线程，在该线程内根据外部输入调整EVS APP的运行状态以及实现具体的图像绘制。在开启图像绘制时，调用getTargetBuffer接口从EVS HAL获取用于显示的Buffer来承载图像绘制的内容，这些buffer统一来源于GraphicBuffer。然后通过调用activate函数来启动 Render，在完成Render前的基础设定后调用drawFrame函数开启绘制。最后完成绘制后调用 returnTargetBufferforDisplay返回显示buffer，如此完成一个循环。

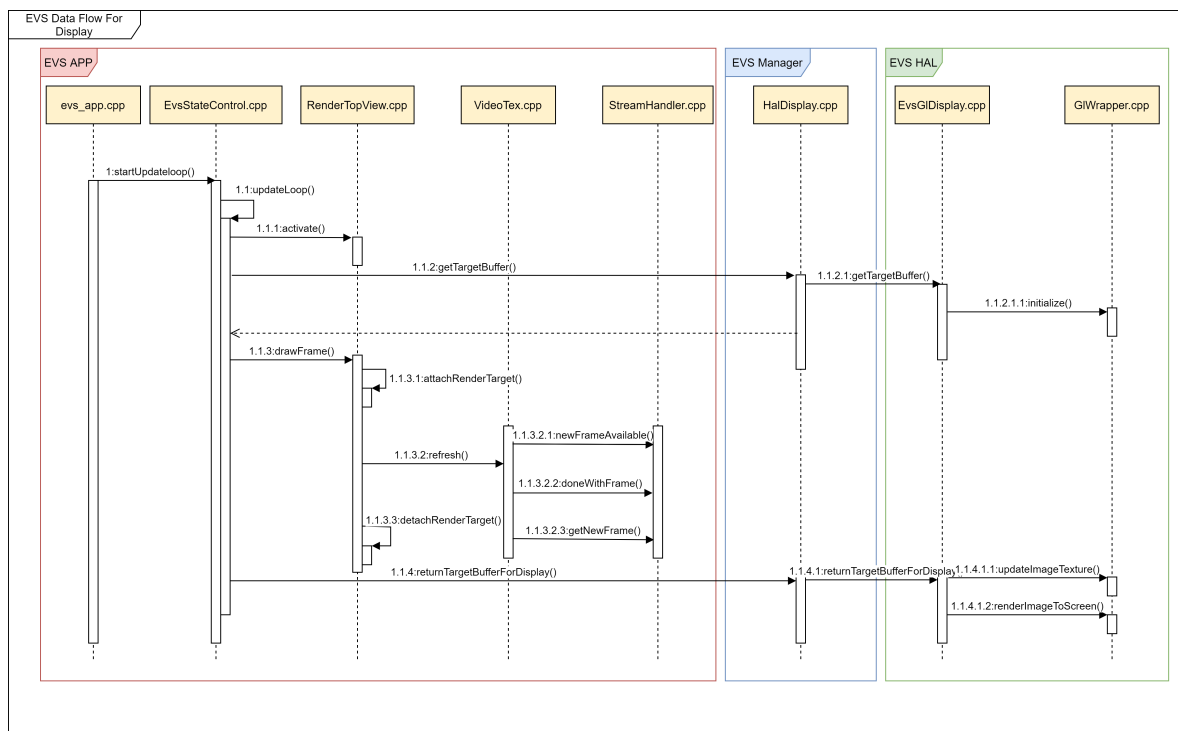


图 EVS Display数据框架流程图

4.4 EVS 产品配置说明

4.4.1 EVS APP产品配置说明

EVS APP的配置位于板端/vendor/etc/automotive/evs/config_override.json，提供了显示配置，摄像头配置等信息，其中摄像头配置中cameraId为EVS HAL传递上来的虚拟摄像头ID，默认0-3给EVS使用。cameraId与 dev/videoX 设备节点的映射关系可查看板端路径 vendor/etc/rvcam_config.xml 文件。此节点属性不可修改

```

{
  "car" : {                                     // This section describes the geometry of the
  car
    "width" : 76.7,                             // The width of the car body
    "wheelBase" : 117.9,                       // The distance between the front and rear
    axel
    "frontExtent" : 44.7,                      // The extent of the car body ahead of the
    front axel
    "rearExtent" : 40                          // The extent of the car body behind the rear
    axel
  },
  "displays" : [                               // This configures the dimensions of the
  surround view display

```

```

    {
        // The first display will be used as the
        default display
        "displayPort" : 1 // Display port number, the target display is
        connected to.
        "frontRange" : 100, // How far to render the view in front of the
        front bumper
        "rearRange" : 100 // How far the view extends behind the rear
        bumper
    }
],
"graphic" : { // This maps the car texture into the
projected view space
    "frontPixel" : 23, // The pixel row in CarFromTop.png at which
the front bumper appears
    "rearPixel" : 223 // The pixel row in CarFromTop.png at which the
back bumper ends
},
"cameras" : [ // This describes the cameras potentially available
on the car
    {
        "cameraId" : "0", // Camera ID exposed by EVS HAL,
        "function" : "reverse,park", // set of modes to which this camera
contributes
        "x" : 0.0, // Optical center distance right of
vehicle center
        "y" : -40.0, // Optical center distance forward of
rear axel
        "z" : 48, // Optical center distance above ground
        "yaw" : 180, // Optical axis degrees to the left of
straight ahead
        "pitch" : -30, // Optical axis degrees above the
horizon
        "roll" : 0, // Rotation degrees around the optical
axis
        "hfov" : 125, // Horizontal field of view in degrees
        "vfov" : 103, // Vertical field of view in degrees
        "hflip" : true, // Flip the view horizontally
        "vflip" : true, // Flip the view vertically
    }
]
}

```

4.4.2 EVS HAL evs_configuration.xml配置说明

EVS HAL产品配置位于板端vendor/etc/automotive/evs/evs_configuration_override.xml，其主要提供了camera信息与现实信息，如下所示，device id为虚拟摄像头id，为唯一的id，不能够重复，默认0-3为EVS使用，其对应的物理摄像头dev/videoX 设备节点的映射关系可查看板端路径vendor/etc/rvcam_config.xml 文件；position只能为front, rear, left, right四个当中的一个；stream里面包含了该虚拟摄像头的分辨率宽高，格式以及帧率信息

```
<camera>
  <!-- vivid emulated video devices -->
  <device id='0' position='rear'>
    <caps>
      <!-- list of supported controls -->
      <supported_controls>
      </supported_controls>
    <!-- list of supported stream configurations -->
    <stream id='0' width='1920' height='1440' format='YCRCB_420_SP'
framerate='30' />
    </caps>

    <!-- list of parameters -->
    <characteristics/>
  </device>
</camera>
```

4.4.3 EVS HAL HIDL 产品配置说明

android.hardware.automotive.evs@1.1-rvcam.rc启动文件配置了服务名称、属性及权限。

```
service evs_rvcam_driver /vendor/bin/hw/android.hardware.automotive.evs@1.1-
rvcam
  class hal
  priority -20
  user graphics
  group automotive_evs camera
  onrestart restart automotive_display
  onrestart restart evs_manager
  disabled # will not automatically start with its class; must be
explicitly started.
```

manifest_android.hardware.automotive.evs@1.1-rvcam，描述了服务接口和版本、binder 通信传输机制及实例名称说明。hidl通过hwbinder通信，其实例接口为IEvsEnumerator/hw/1

```

<manifest version="1.0" type="device" >
  <hal format="hidl">
    <name>android.hardware.automotive.evs</name>
    <transport>hwbinder</transport>
    <version>1.1</version>
    <interface>
      <name>IEvsEnumerator</name>
      <instance>hw/1</instance>
    </interface>
  </hal>
</manifest>

```

除此之外，还需要给 EVS HAL 添加相关的 selinux 配置。首先在 file_contexts 给该服务添加标签。

```

/vendor/bin/hw/android\hardware\automotive\evs@1\.[0-9]+-rvcam
u:object_r:hal_evs_driver_exec:s0

```

其次在 evs_driver.te 中添加其他进程的权限

```

evs reference hardware driver service

hal_server_domain(hal_evs_driver, hal_evs)
hal_client_domain(hal_evs_driver, hal_evs)

allow init to launch processes in this context

type hal_evs_driver_exec, exec_type, file_type, vendor_file_type;
init_daemon_domain(hal_evs_driver)
binder_use(hal_evs_driver)

binder_call(hal_evs_driver hal_rvcam_default);
binder_call(hal_rvcam_default hal_evs_driver);

Allow use of USB devices, gralloc buffers, and surface flinger

allow hal_evs_driver device:dir { open read };
allow hal_evs_driver video_device:chr_file rw_file_perms;
hal_client_domain(hal_evs_driver, hal_graphics_allocator);
hal_client_domain(hal_evs_driver, hal_graphics_composer)
hal_client_domain(hal_evs_driver, hal_configstore)
... ..

```

将模块加入系统编译：

```
PRODUCT_PACKAGES += \
    evs_app \
    evs_manager \
    android.hardware.automotive.evs@1.1-rcam \
    android.frameworks.automotive.display@1.0-service
```

HIDL接口版本使用的显示服务是android.frameworks.automotive.display@1.0-service服务，EVS Manager服务也为HIDL接口的evs_manager服务

4.4.4 EVS HAL AIDL 产品配置说明

android.hardware.automotive.evs-rcam.rc启动文件配置了服务名称、属性及权限。

```
service evs_rcam_driver /vendor/bin/hw/android.hardware.automotive.evs-rcam
    class early_hal
    priority -20
    user graphics
    group automotive_evs camera
    onrestart restart cardisplayproxymd
    onrestart restart evsmanagerd
    disabled
```

manifest_android.hardware.automotive.evs-rcam.xml与HIDL版本除了接口改为AIDL外，其名称与实例并无差别

```
<manifest version="1.0" type="device">
    <hal format="aidl">
        <name>android.hardware.automotive.evs</name>
        <fqname>IEvsEnumerator/hw/1</fqname>
    </hal>
</manifest>
```

除此之外，AIDL版本的selinux配置与HIDL版本差异较大，AIDL版本直接使用系统的selinux配置，其目录在system/sepolicy/vendor/hal_evs_default.te

将模块加入系统编译：

```
PRODUCT_PACKAGES += \
    evs_app \
    evsmanagerd \
    android.hardware.automotive.evs-rcam \
    cardisplayproxymd
```

AIDL接口版本使用的显示服务为cardisplayproxyd，使用的EVS 管理服务为evsmanagerd。

5. HAL3

5.1 基本功能说明

Automotive Camera HAL3 (以下简称 AuthAL3) 是用于连接相机框架 API 和 RVCAM 中间件的硬件抽象层，提供了对图像处理流程的支持，包括预览、捕获、录像等功能。AuthAL3 的引入是为了进一步增强相机功能，提高性能，并为开发者提供更多的控制权。它包括对相机设备的更细粒度控制，以及对新的相机硬件特性的支持，例如适配多摄像头系统、高级图像处理等功能。

Camera 系统框架图如下所示，相机应用可选择 Camera API1 或 Camera API2 方法访问相机服务。框架层的 CameraService 负责协调不同应用对相机资源的请求，通过 ICameraDevice、ICameraProvider 等 HIDL/AIDL 接口与AutomotiveCamera HAL3 实现通信。与 ExternalCamera HAL3 不同的是，ExternalCamera HAL3 向下直接对接 V4L2 硬件驱动，而 AutomotiveCamera HAL3 则作为相机框架层和 RVCAM 虚拟多摄中间件的桥梁，RVCAM 与物理摄像头交互并扩展出多个虚拟摄像头。这种机制允许开发者构建复杂的相机应用，同时保持对相机硬件的高效控制。

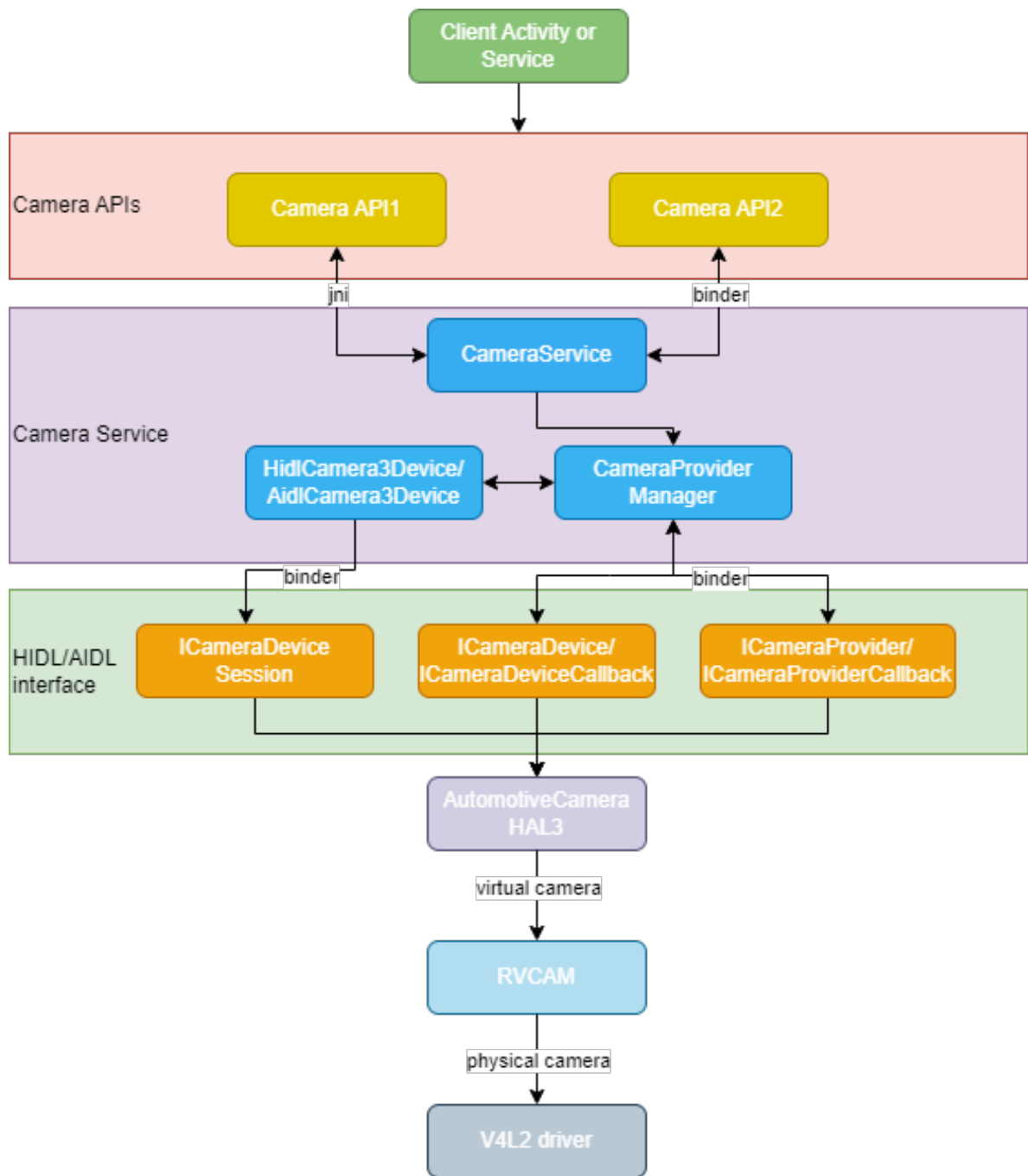


图 Camera 系统框架图

5.1.1 AutomotiveCamera HAL3 流程介绍

框架层可以通过 `ICameraDevice::open` 打开摄像头设备，HAL3 会创建绘画并调用 `rvcam_open` 打开 RVCAM 中间件。框架层可随时调用 `ICameraDeviceSession::configureStreams()` 配置输入/输出流，为后续的捕获请求默认配置，同时会话通过 `rvcam_start` 通知 RVCAM 开流。配置之后可以向 HAL3 发出单次或连续的捕获请求，可以是拍照、预览、录像等请求。待请求结束之后，框架层调用 `ICameraDevice::close` 结束此次流程，HAL3 也依次调用 `rvcam_stop` 和 `rvcam_close` 关

闭。

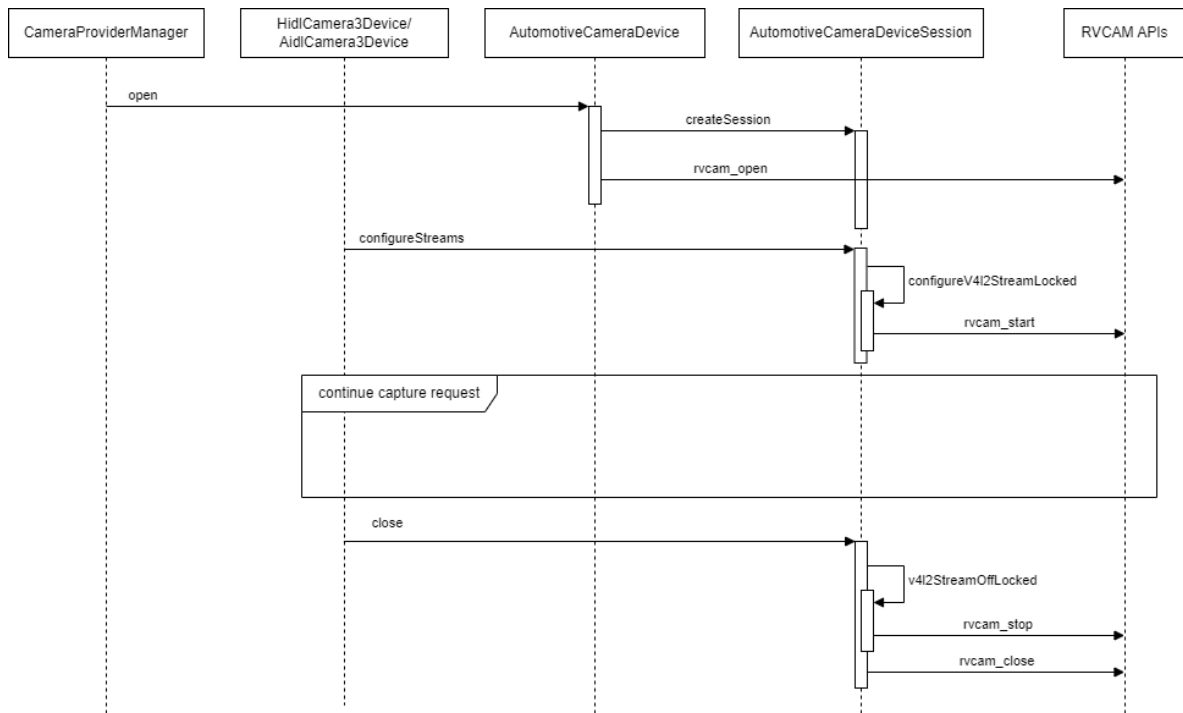


图 基本流程图

具体的捕获请求流程如图 3 所示，框架层通过调用

`ICameraDeviceSession::processCaptureRequest()` 发起捕获请求，一次可发起多个请求。HAL3收到捕获请求后，等待 RVCAM 中间件通知 buffer 已经准备好，通过 `rvcam_frame_info_t` API 获取帧信息。HAL3 会按照先前的输入/输出流配置填充输出缓冲区，调用 `ICameraDeviceCallback::notify()` 返回 SHUTTER 消息，包括帧号和开始曝光的时间戳。其次，将捕获结果通过 `ICameraDeviceCallback::processCaptureResult()` 返回到框架层。一次捕获结束之后，HAL3 会将 buffer 通过 `rvcam_release_frame` 返还给 RVCAM 中间件。若框架层发起多个请求，HAL3 会确保捕获请求的有序处理和结果的按序返回。

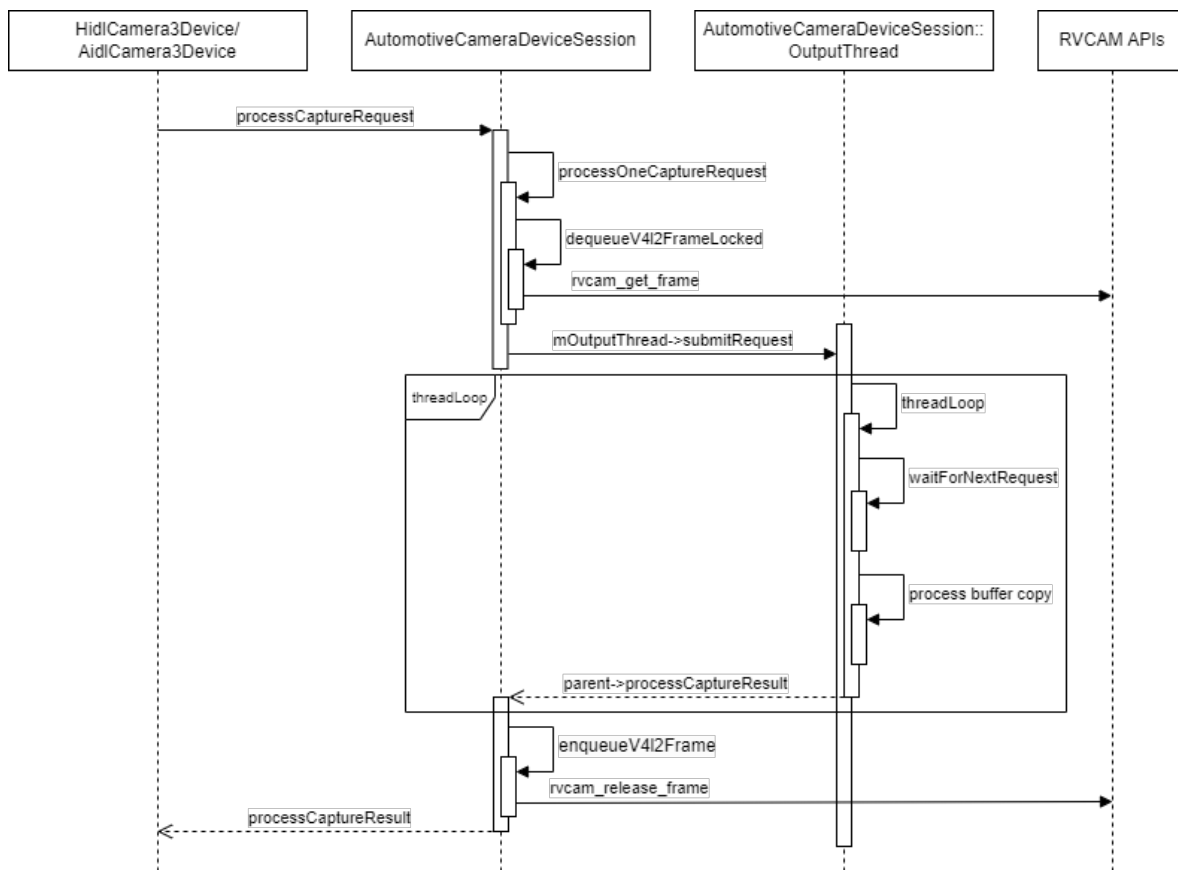


图 捕获请求流程图

5.1.2 AutomotiveCamera HAL3 Provider 版本

hardware/interfaces/camera/provider 中包含了 ICameraProvider@2.4~2.7 HIDL 接口及 ICameraProvider@V1 AIDL 接口，版本的使用会随着新接口适配、Android 系统迭代而更新。为确保与 Android 版本兼容，HAL3 在 Android 14.0 以下本使用 ICameraProvider@2.4 HIDL 接口，Android 14.0 及其更高版本上使用 ICameraProvider@V1 AIDL 接口。

ICameraProvider.hal 和 ICameraProviderCallback.hal 接口用于管理 camera 设备并向框架层服务上报其连接状态。相机服务通过 ICameraProvider.hal 接口请求相机设备的列表、创建相机设备实例、以及管理相机设备的生命周期。ICameraProviderCallback 接口允许相机服务注册回调，以便在相机设备列表更新、相机设备连接或断开时得到通知。这使得相机服务能够动态地响应系统变化，例如更换了外部相机设备。

5.1.3 AutomotiveCamera HAL3 Device 版本

hardware/interfaces/camera/device 中包含了 ICameraDevice@1.0、ICameraDevice@3.2~3.7 HIDL 接口及 AIDL 接口，HAL3 在 Android 14.0 以下本使用 ICameraDevice@3.5 HIDL 接口，Android 14.0 及其更高版本上使用 ICameraDevice@V1 AIDL 接口。

ICameraDevice.hal 接口提供了相机设备的基本操作，如配置相机参数、创建和销毁会话。

ICameraDeviceCallback.hal 接口定义了相机设备回调，返回 SHUTTER 消息和捕获结果。

ICameraDeviceSession.hal 接口用于管理相机设备的会话，配置相机数据流参数，以及在会话中执行具体的捕获请求。

相较于 ICameraDevice@3.4 版本接口，ICameraDevice@3.5版本新增了 Buffer Management 相关 API，灵活管理 buffer 缓冲区的申请和返回。ICameraDevice@V1 AIDL 接口默认增加并实现 Buffer Management 相关 API。

5.1.4 automotive_camera_config.xml 配置说明

automotive_camera_config.xml 定义了 AutomotiveCamera HAL3 的摄像头数量、能力参数及特定功能开关等，部分节点可供修改和配置。

`ignore` 和 `CameraIdOffset` 节点规定了 Automotive camera 起始 id，0~3 号 camera id 专用于 EVS HAL，HAL3 使用的 camera id 从 4 号开始。此处的 camera id 指的是虚拟摄像头序列号，与 dev/videoX 设备节点的映射关系可查看板端路径 vendor/etc/rvcam_config.xml 文件。此节点属性不可修改。

```
<Provider>
  <ignore> <!-- Internal video devices to be ignored by external camera
HAL -->
  <!-- <id>0</id> --> <!-- No leading/trailing spaces -->
  <id>0</id>
  <id>1</id>
  <id>2</id>
  <id>3</id>
</ignore>
<CameraIdOffset>
  -4
</CameraIdOffset>
</Provider>
```

`BufferManager` 节点是 Android 14.0 及以上系统的 HAL3 的 Buffer Management 功能开关，默认情况下为关闭，可根据使用场景选择开启或关闭该功能。Android 14.0 以下系统默认开启该功能。

```
<BufferManager enabled="false"/>
```

`HalBufferTransfer` 节点定义了 Android 12.0 系统上的 HAL3 buffer 透传功能开关，具体功能介绍详见《buffer 透传功能》章节。默认情况下为开启，可根据应用场景选择开启或关闭该功能。

```
<HalBufferTransfer enabled="true"/>
```

5.2 新增功能说明

相对于 ExternalCamera HAL3, AutomotiveCamera HAL3 升级了 Buffer Management 功能, 用于管理 buffer 缓冲区。此外, AutomotiveCamera HAL3 新增了 buffer 透传功能, 将框架层捕获请求的输出 buffer 直接传输给 RVCAM 中间件, 节省虚拟摄像头和物理摄像头交互的中间内存。

5.2.1 BUFFER MANAGEMENT 功能

Buffer Management 是从 Android 10 开始推出的一个可选的内存管理方式, 旨在 HAL 层可以在需要填充图形数据时才向框架层申请 buffer, 以达到减少 buffer 使用峰值、让 capture request 更快下发的目的。

实现 Buffer Management 功能, 首先要将

`ANDROID_INFO_SUPPORTED_BUFFER_MANAGEMENT_VERSION` 元数据改为 `ANDROID_INFO_SUPPORTED_BUFFER_MANAGEMENT_VERSION_HIDL_DEVICE_3_5`。框架层可依据该元数据的值判断是否在下发 capture request 时携带 buffer, 若为 `HIDL_DEVICE_3_5` 则不携带 buffer。因此框架层不必等到 buffer pool 有可用 buffer, 在 capture request 限制数量之内可直接、快速下发捕获请求。

其次是实现 `ICameraDevice@3.5` 和 `ICameraDevice@V1` 所增加的接口

`ICameraDeviceCallback::requestStreamBuffers` 和 `ICameraDeviceCallback::returnStreamBuffers`, 分别用于请求和释放 buffer。在捕获流程中, 回调 `ICameraDeviceCallback::processCaptureResult()` 接口便可以向框架层依次返回 buffer。`ICameraDeviceCallback::returnStreamBuffers` 接口一般使用在 `ICameraDeviceSession::configureStreams()` 前或 `ICameraDeviceSession::flush` 时, 一次性回收全部 buffer。

如果不使用缓冲区管理 API, 传输框架会针对每个捕获请求输出缓冲区, 然后再向 HAL 层发送捕获请求。如果使用缓冲区管理 API, 则摄像头传输框架不再需要等待缓冲区请求完成, 因此可以更早向相机 HAL 发送捕获请求, 待到实际使用时才根据数量申请 buffer, 流程如下图所示。

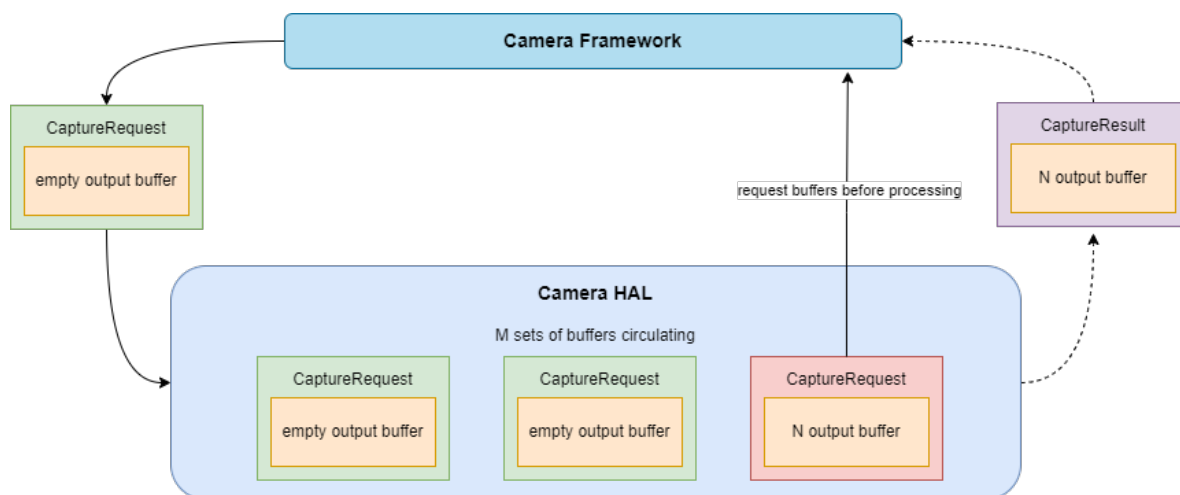


图4 buffer management 示例图

5.2.2 BUFFER 透传功能

AutomotiveCamera HAL3 比 ExternalCamera HAL3 多了一层 RVCAM 中间件的处理流程，AutomotiveCamera HAL3 会额外申请中间 buffer 用于中转和传输图像数据，因此单帧数据从 V4L2 硬件驱动到 HAL 层需要经过两次数据处理和拷贝，如下图所示。

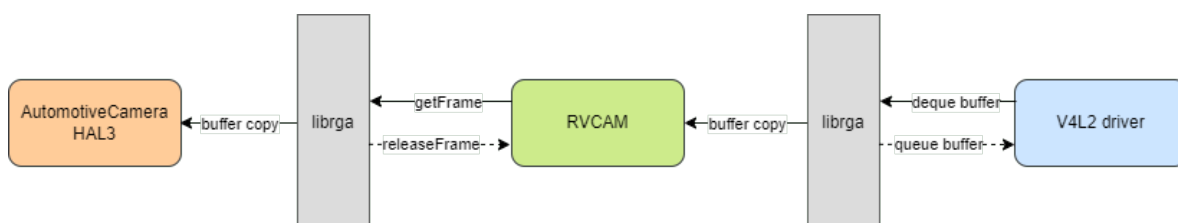


图5 非透传模式数据传递示例图

buffer 透传功能旨在将 capture request 中的 output buffer handle 直接透传给 RVCAM，由 RVCAM 将 V4L2 帧数据拷贝给 output buffer，可以减少中间内存，RGA 处理的次数也能减半。

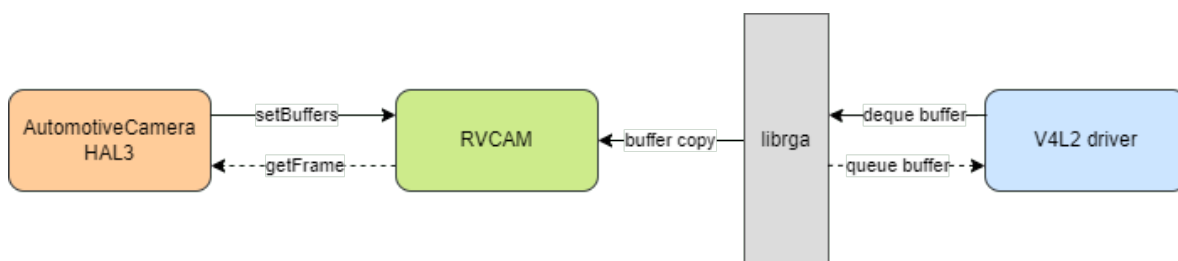


图5 透传模式数据传递示例图

5.3 性能指标说明

item	describe
虚拟摄像头数量	参考板端路径 vendor/etc/rvcam_config.xml 板端信息
每路摄像头支持的 tall stream 数量	1
每路摄像头支持的其他 stream 数量	2
支持的图像格式	HAL_PIXEL_FORMAT_BLOB/ HAL_PIXEL_FORMAT_YCbCr_422_I/ HAL_PIXEL_FORMAT_YCbCr_420_888/ HAL_PIXEL_FORMAT_IMPLEMENTATION_DEFINED
支持的分辨率（以物理摄像头参数为准）	1920×1440/1440×1080/ 1280×960/960×720/640×480
支持的帧率（以物理摄像头参数为准）	30fps

[^注]： **tall stream** 指的是输出图像格式为 HAL_PIXEL_FORMAT_BLOB 的流。

5.4 调试功能说明

5.4.1 CAMERA TRACE 调试功能

Perfetto 是一个由 Google 开发的高性能、可扩展的跨平台跟踪系统，用于收集、记录和分析系统和应用程序的性能数据。Perfetto 支持在 Android 设备上抓取 trace 数据，以便开发者和系统管理员能够深入分析系统性能和稳定性问题。

首先需要开启内核编译 `CONFIG_FTRACE`

```
CONFIG_FTRACE=y
```

使用 Perfetto 命令抓取 camera 模块相关 trace，举例如下：

```
adb shell perfetto \
  -c - --txt \
  -o /data/misc/perfetto-traces/trace \
  <<EOF

buffers: {
  size_kb: 63488
  fill_policy: DISCARD
}
```



```

buffers: {
    size_kb: 2048
    fill_policy: DISCARD
}
data_sources: {
    config {
        name: "android.packages_list"
        target_buffer: 1
    }
}
data_sources: {
    config {
        name: "linux.ftrace"
        ftrace_config {
            ftrace_events: "ftrace/print"
            atrace_categories: "camera"
            atrace_apps: "*"
        }
    }
}
duration_ms: 10000

EOF

```

使用 <https://ui.perfetto.dev/> 网页查看 trace 文件，分析相机帧率、时延等信息。

5.4.2 查看相机配置参数

\$ adb shell dumpsys media.camera 命令可以查询到相机元数据，包括支持的图像格式、分辨率、帧率等参数。例如 android.scaler.availableStreamConfigurations 元数据存储的是输入或输入流的可选图像格式和对应分辨率，android.scaler.availableMinFrameDurations 元数据存储的是不同图像格式和分辨率对应的帧间隔。

5.5 产品配置说明

5.5.1 AutomotiveCamera HAL HIDL 实例配置说明

android.hardware.camera.provider@2.4-automotive-service.rc 启动文件配置了服务名称、属性及权限。

```

service vendor.camera-provider-2-4-aut
    /vendor/bin/hw/android.hardware.camera.provider@2.4-automotive-service
    # 制定了实现服务的硬件接口，实例名称为 automotive/0

```

```

interface android.hardware.camera.provider@2.4::ICameraProvider
automotive/0
# 该服务属于 HAL 类型
class hal
# 该服务的用户和组权限
user cameraserver
group audio camera input drmrpc usb
# 设置该服务的 I/O 优先级为实时调度等级 4
ioprio rt 4
# 赋予该服务 SYS_NICE 能力
capabilities SYS_NICE
# 指定该服务的任务配置文件，可以控制服务的资源分配和性能优化
task_profiles CameraServiceCapacity MaxPerformance

```

框架兼容性矩阵 FCM，在不同的安卓版本中声明 HAL 版本范围，实现 HAL 的版本控制。DCM 和 FCM 旨在 OTA 更新时进行协调，以验证框架和供应商实现是否可以相互协同工作，因此这两份清单所声明的 HAL 版本必须匹配。rvcam_compatibility_matrix_s.xml 规定了 android 12 系统上的 Camera HAL 版本范围为 2.4~2.7，实例名称用正则表达式表示为 任意字符/(0-9)。

```

<hal format="hidl" optional="true">
  <name>android.hardware.camera.provider</name>
  <version>2.4-7</version>
  <interface>
    <name>ICameraProvider</name>
    <regex-instance>[^/]+/[0-9]+</regex-instance>
  </interface>
</hal>

```

android.hardware.camera.provider@2.4-automotive.xml 设备兼容性矩阵 DCM 文件，描述了服务接口和版本、binder 通信传输机制及实例名称说明，这里声明了两个实例 legacy/0 和 automotive/0。实际在注册服务时，使用的是 automotive/0 实例。

```

<manifest version="1.0" type="device">
  <hal format="hidl">
    <name>android.hardware.camera.provider</name>
    <transport>hwbinder</transport>
    <version>2.4</version>
    <interface>
      <name>ICameraProvider</name>
      <instance>legacy/0</instance>
      <instance>automotive/0</instance>
    </interface>
  </hal>
</manifest>

```

除此之外，还需要给 Camera HAL 添加相关的 selinux 配置。首先在 file_contexts 给该服务添加标签。

```
/(vendor|system/vendor)/bin/hw/android\.hardware\.camera\.provider@2\.[0-9]+-automotive-service          u:object_r:hal_camera_default_exec:s0
```

其次在 hal_camera_default.te 文件中添加对其他进程或文件的访问权限。

```
allow hal_camera_default vndbinder_device:chr_file rw_file_perms;
allow hal_camera_default hal_graphics_mapper_hwservice:hwservice_manager {
    find };

allow hal_camera_default hal_graphics_composer_default:fd { use };
allow hal_camera_default mediaserver:fd { use };
allow hal_camera_default vendor_data_file:dir { read open write add_name
    create setattr getattr remove_name rmdir };
allow hal_camera_default vendor_data_file:file { read open write getattr
    create};
... ..
```

将模块加入系统编译：

```
PRODUCT_PACKAGES += \
    android.hardware.camera.provider@2.4-impl-automotive \
    android.hardware.camera.provider@2.4-automotive-service
```

5.5.2 AutomotiveCamera HAL AIDL 实例配置说明

android 14 系统的 Camera HAL 服务还提供了懒启动方式，只有服务被请求时才会启动，且只会启动一次。若要使用懒启动方式，需要在 Android.bp 构建服务时指定 `init_rc` 选项为 `android.hardware.camera.provider-V1-automotive-service.rc`。

```
// android.hardware.camera.provider-V1-automotive-service.rc
service vendor.camera.provider-aut
/vendor/bin/hw/android.hardware.camera.provider-V1-automotive-service
    interface aidl
android.hardware.camera.provider.ICameraProvider/automotive/0
    class hal
    user cameraserver
    group audio camera input drmrpc usb
    ioprio rt 4
    capabilities SYS_NICE
    task_profiles CameraServiceCapacity MaxPerformance
```

```
// android.hardware.camera.provider-V1-automotive-service-lazy.rc
service vendor.camera.provider-aut
/vendor/bin/hw/android.hardware.camera.provider-V1-automotive-service-lazy
    interface aidl
android.hardware.camera.provider.ICameraProvider/automotive/0
    class hal
    oneshot
    disabled
    user camerasetter
    group audio camera input dmrpc usb
    ioprio rt 4
    capabilities SYS_NICE
    task_profiles CameraServiceCapacity MaxPerformance
```

rvcam_compatibility_matrix_u.xml 规定了android 14 系统上的 Camera HAL 版本范围为 V1~V2，实例名称用正则表达式表示为 任意字符/(0-9)。

```
<hal format="aidl" optional="true">
  <name>android.hardware.camera.provider</name>
  <version>1-2</version>
  <interface>
    <name>ICameraProvider</name>
    <regex-instance>[^/]+/[0-9]+</regex-instance>
  </interface>
</hal>
```

android.hardware.camera.provider-V1-automotive.xml 声明实例 automotive/0。

```
<manifest version="1.0" type="device">
  <hal format="aidl" >
    <name>android.hardware.camera.provider</name>
    <version>2</version>
    <interface>
      <name>ICameraProvider</name>
      <instance>automotive/0</instance>
    </interface>
  </hal>
</manifest>
```

相应的 selinux 标签声明如下：

```
// file_contexts
/(vendor|system/vendor)/bin/hw/android\.hardware\.camera\.provider-V1-
automotive-service          u:object_r:hal_camera_default_exec:s0

// service_contexts
/(vendor|system/vendor)/bin/hw/android\.hardware\.camera\.provider@2\.[0-
9]+-automotive-service      u:object_r:hal_camera_default_exec:s0
```

对其他进程或文件的访问权限声明在 hal_camera_default.te 中

```
allow hal_camera_default vndbinder_device:chr_file rw_file_perms;
allow hal_camera_default hal_graphics_mapper_hwservice:hwservice_manager {
find };

allow hal_camera_default hal_graphics_composer_default:fd { use };
allow hal_camera_default mediaserver:fd { use };
allow hal_camera_default vendor_data_file:dir { read open write add_name
create setattr getattr remove_name rmdir };
allow hal_camera_default vendor_data_file:file { read open write getattr
create};
allow hal_camera_default sysfs:dir {read open};
allow hal_camera_default sysfs:file {read open getattr};
allow hal_camera_default sysfs_camera:file rw_file_perms;
allow hal_camera_default hal_hdmi_hwservice:hwservice_manager { find };
... ..
```

将模块加入系统编译：

```
PRODUCT_PACKAGES += \
    android.hardware.camera.provider-V1-automotive-impl \
    android.hardware.camera.provider-V1-automotive-service
```

5.6 单应用多开、多应用多开示例

rvcam_config.xml 文件描述了物理摄像头参数、物理摄像头和虚拟摄像头的映射关系，存储于板端 vendor/etc 路径下，板级配置有所不同。以 Rockchip RK3588 VEHICLE EVB V21 Board 配置为例，物理摄像头与虚拟摄像头映射数量默认配置一对三，前四个虚拟摄像头固定为 EVS HAL 使用。

```
<board name="Rockchip RK3588 VEHICLE EVB V21 Board">
  <cameras>
    <camera id="0">
      <entity>m00_b_max96712 6-0029</entity>
      <subDev>/dev/v4l-subdev5</subDev>
```

```
<mbusCode>UYVY2X8/1920x1440</mbusCode>
<channels>
  <channel id="0" dev="/dev/video0">
    <cameraModel>TXD</cameraModel>
    <stream id="0" format="YUYV" width="1920" height="1440"
bufCnt="4"/>
  </channel>
  <channel id="1" dev="/dev/video1">
    <cameraModel>TXD</cameraModel>
    <stream id="0" format="YUYV" width="1920" height="1440"
bufCnt="4"/>
  </channel>
  <channel id="2" dev="/dev/video2">
    <cameraModel>TXD</cameraModel>
    <stream id="0" format="YUYV" width="1920" height="1440"
bufCnt="4"/>
  </channel>
  <channel id="3" dev="/dev/video3">
    <cameraModel>TXD</cameraModel>
    <stream id="0" format="YUYV" width="1920" height="1440"
bufCnt="4"/>
  </channel>
</channels>
</camera>
</cameras>
<inputMapping>
  <inputMap camId="0">
    <inputSrc cameraId="0" channelId="0" streamId="0"/>
  </inputMap>
  <inputMap camId="1">
    <inputSrc cameraId="0" channelId="1" streamId="0"/>
  </inputMap>
  <inputMap camId="2">
    <inputSrc cameraId="0" channelId="2" streamId="0"/>
  </inputMap>
  <inputMap camId="3">
    <inputSrc cameraId="0" channelId="3" streamId="0"/>
  </inputMap>
  <inputMap camId="4">
    <inputSrc cameraId="0" channelId="0" streamId="0"/>
  </inputMap>
  <inputMap camId="5">
    <inputSrc cameraId="0" channelId="1" streamId="0"/>
  </inputMap>
  <inputMap camId="6">
    <inputSrc cameraId="0" channelId="2" streamId="0"/>
  </inputMap>
  <inputMap camId="7">
    <inputSrc cameraId="0" channelId="3" streamId="0"/>
  </inputMap>
</inputMapping>
```

```

</inputMap>
<inputMap camId="8">
  <inputSrc cameraId="0" channelId="0" streamId="0"/>
</inputMap>
<inputMap camId="9">
  <inputSrc cameraId="0" channelId="1" streamId="0"/>
</inputMap>
<inputMap camId="10">
  <inputSrc cameraId="0" channelId="2" streamId="0"/>
</inputMap>
<inputMap camId="11">
  <inputSrc cameraId="0" channelId="3" streamId="0"/>
</inputMap>
</inputMapping>
</board>

```

以 Native2 apk 为例，应用布局最高可以显示 16 个虚拟摄像头画面，使用以上配置可以打开 8 个摄像头。如需显示 12 个虚拟摄像头画面，rvcam_config.xml 文件应在此基础上增加 4 个虚拟通路，如下所示：

```

... ..
<inputMap camId="12">
  <inputSrc cameraId="0" channelId="0" streamId="0"/>
</inputMap>
<inputMap camId="13">
  <inputSrc cameraId="0" channelId="1" streamId="0"/>
</inputMap>
<inputMap camId="14">
  <inputSrc cameraId="0" channelId="2" streamId="0"/>
</inputMap>
<inputMap camId="15">
  <inputSrc cameraId="0" channelId="3" streamId="0"/>
</inputMap>
</inputMapping>
</board>

```

6. RVCAM 多系统支持

在 Android 单系统中，Automotiv Camera HAL3、EVS HAL 和其他应用均使用 Binder 通信与 RVCAM 中间件进行数据交互。在多系统环境中，由于硬件资源限制，Android 应用和 RVCAM 中间件可能运行在不同的系统上，需要依赖 RPC 机制实现数据传输。以下多系统通信情况以 Android 和 Linux 双系统举例说明。

在 Android + Linux 双系统中，RVCAM 中间件运行在 Linux 系统上，与 V4L2 driver 和 DRM driver 交互，而 Android 系统上的 HAL 服务以及其他应用无法直接与 Linux 系统上的 RVCAM 中间件进行通信。双系统通信选择 RPC 进行数据传输。双系统的数据传输路径如下图所示：

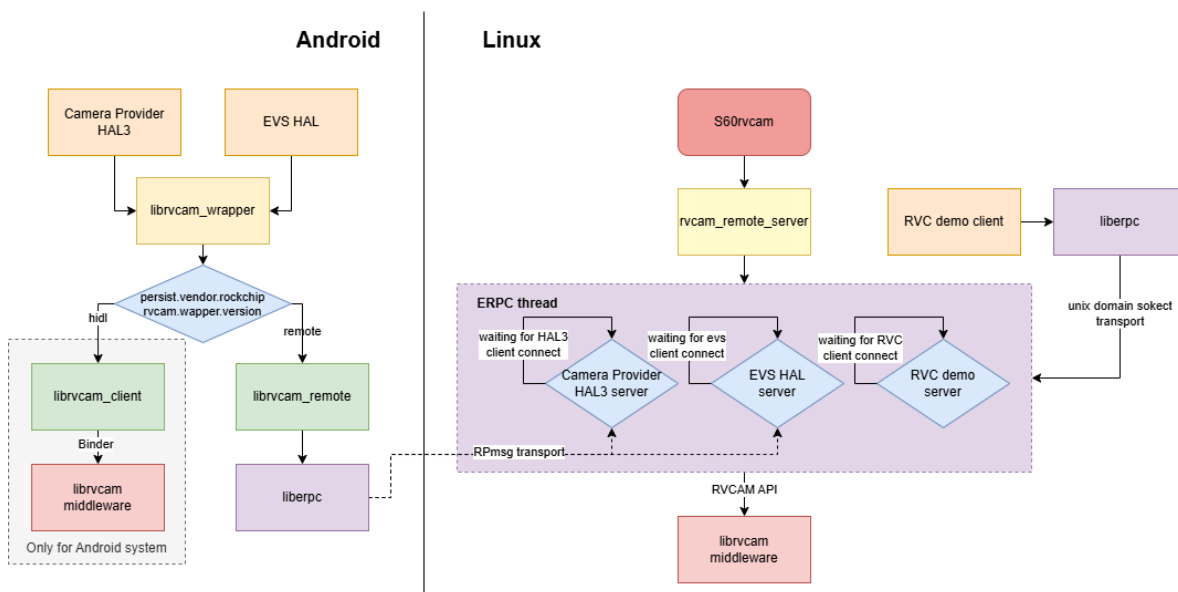


图 RVCAM 双系统框图

6.1 eRPC 简介

eRPC 是一个开源的远程过程调用系统,专门为嵌入式多芯片系统和异构多核 SoC 设计。本系统在 eRPC 开源代码基础上，适配 RK3576M 硬件架构，实现跨系统 RPC 通信。

eRPC 的 IDL 脚本 erpcgen 可以根据自定义 .erpc 文件，生成 client 和 server 的 C/C++、Java、Python 接口定义和通信代码。这些代码包括数据结构、函数声明、以及用于序列化和反序列化数据的代码。生成的代码可以直接在嵌入式系统中使用，以实现高性能的通信。

RVCAM 中间件在硬隔离双系统上使用的 eRPC 传输层协议有 RPMSG 和 Unix Domain Socket 两种。

- RPMSG 传输层基于 virtio 框架，在硬隔离双系统中将 Android 系统作为 RPmsg server 端，Linux 系统作为 RPMSG client 端，host 端通过共享内存向 remote 传输数据或从 remote 接收数据。
- Unix Domain Socket 传输层使用标准 Unix Domain Socket 协议进行数据传输，不需要经过网络协议栈，使用其目的是为了单系统跨进程传递文件描述符。

eRPC 通信的数据长度有限制，目前可供使用的大小为 1024 byte，具体上限值由传输层通信协议决定。

6.2 RVCAM 跨系统通信实现

6.2.1 RVCAM remote 模块实现

使用 eRPC 实现了 RVCAM remote 对应的 client 和 server 端。在硬隔离双系统中，Android 端使用 RVCAM remote client 序列化代码和 RVCAM API 接口集成的 librvcam_remote.so 库，Camera Povidter HAL3 和 EVS HAL 分别通过 librvcam_wrapper 选择性动态加载 librvcam_remote.so，使用不同的 RPmsg 端口发起连接和请求。Linux 端使用 RVCAM remote server 序列化代码集成 rvcam_remote_server 服务，开机后自启动，等待多个 client 端的连接和请求。S60rvcam 脚本为 rvcam_remote_server 提供了 stop 和 start 命令，用于控制其启动和停止。server 端和 client 端使用 eRPC RPMSG 传输层, buffer 传输使用 CMA 共享内存。

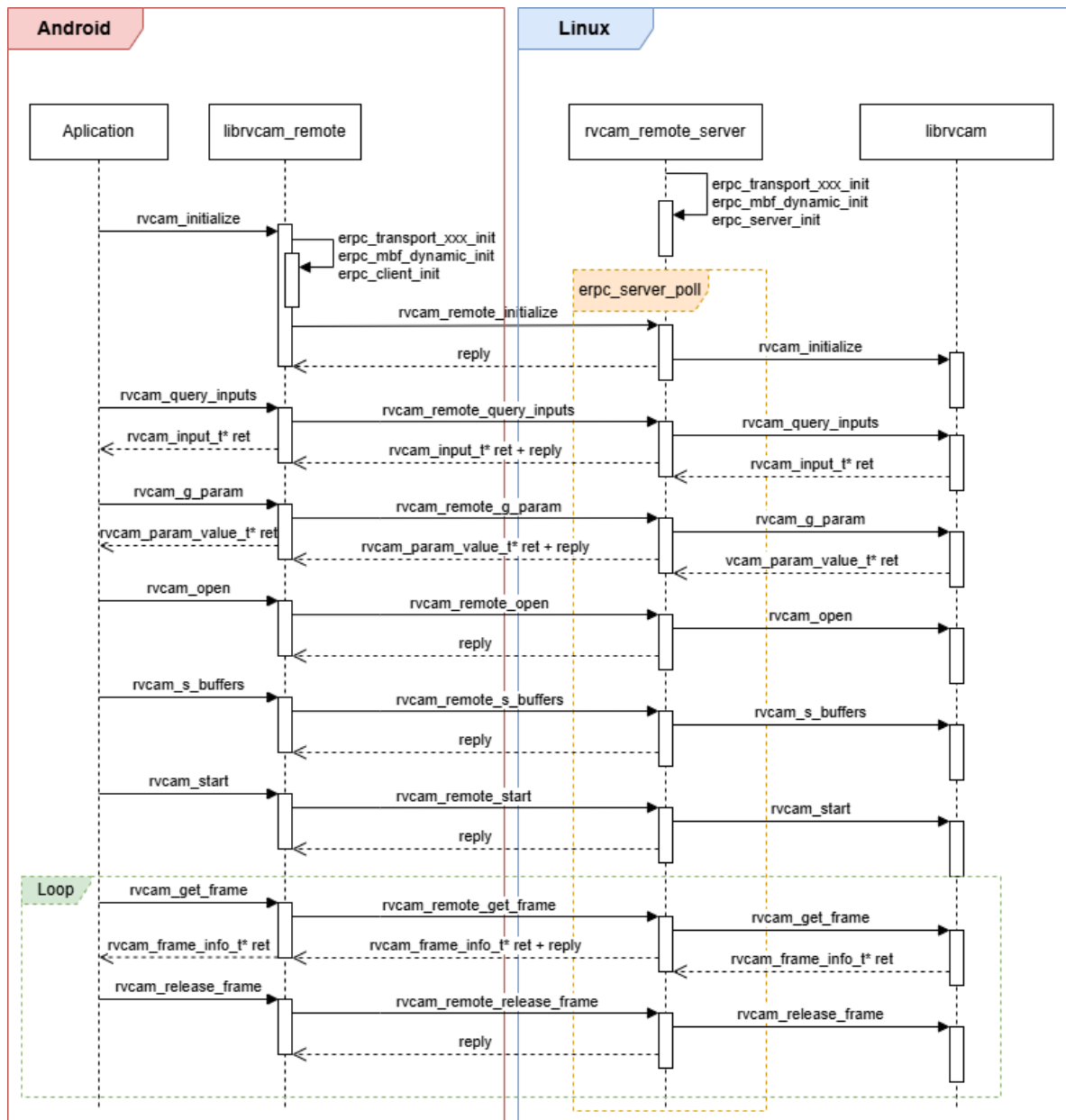


图 RVCAM remote 调用流程

同时，RVC demo 应用程序跑在 Linux 系统上，目前只是显示单摄影像。使用 unix domain socket 传输层，其建立连接和发送请求的流程和上述类似。所有的 client 应用需要对接同一个 rvcam_remote_server 服务，旨在由 librvcam 中间件统一管理物理摄像头的开关出流和虚拟摄像头实例的创建和销毁。

6.2.2 RVCAM remote 接口说明

以 rvcam_remote 为前缀的接口是由 IDL 接口生成，主要功能是作为 librvcam_remote 和 librvcam 的数据中转。librvcam_remote 接口遵循 RVCAM API 声明，功能与 RVCAM API 说明基本一致，个别接口有些许差异，使用到的接口简要说明如下：

- **rvcam_initialize**

```
int rvcam_initialize(rvcam_init_t* p_init_params);
```

功能描述：初始化 ePRC 传输层，调用 rvcam_remote_initialize 连接到中间件并初始化上下文。

- **rvcam_uninitialize**

```
int rvcam_uninitialize(void);
```

功能描述：销毁 ePRC 传输层，调用 rvcam_remote_uninitialize 断开与中间件的连接并释放上下文。

- **rvcam_query_inputs**

```
int rvcam_query_inputs(rvcam_input_t* p_inputs,  
                      unsigned int size,  
                      unsigned int* ret_size);
```

功能描述：调用 rvcam_remote_query_inputs 从中间件获取所有虚拟设备的输入信息。

- **rvcam_g_param**

```
int rvcam_g_param(rvcam_hndl_t hndl,  
                 rvcam_param_t param,  
                 rvcam_param_value_t* p_value);
```

功能描述：调用 rvcam_remote_g_param 从中间件获取虚拟摄像头参数值。

- **rvcam_s_param**

```
int rvcam_s_param(rvcam_hndl_t hndl,
                 rvcam_param_t param,
                 const rvcam_param_value_t* p_value);
```

功能描述：调用 `rvcam_remote_s_param` 设置中间件虚拟摄像头参数值。

- **rvcam_open**

```
rvcam_hndl_t rvcam_open(int id);
```

功能描述：调用 `rvcam_remote_open` 创建虚拟摄像头实例，打开映射的物理摄像头流，返回虚拟摄像头句柄。

- **rvcam_close**

```
int rvcam_close(rvcam_hndl_t hndl);
```

功能描述：调用 `rvcam_remote_close` 销毁虚拟摄像头实例，关闭映射的物理摄像头流。

- **rvcam_s_buffers**

```
int rvcam_s_buffers(rvcam_hndl_t hndl,
                   rvcam_buffers_t* p_buffers);
```

功能描述：调用 `rvcam_remote_s_buffers` 传递 buffer 队列给中间件，如果是跨系统通信，传递的是 buffer 的物理地址，如果使用 unix domain socket 同系统传输，传递的是 buffer 的文件描述符。

- **rvcam_start**

```
int rvcam_start(rvcam_hndl_t hndl);
```

功能描述：调用 `rvcam_remote_start` 启动与句柄关联的摄像头流。

- **rvcam_stop**

```
int rvcam_stop(rvcam_hndl_t hndl);
```

功能描述：调用 `rvcam_remote_stop` 停止与句柄关联的摄像头流。

- **rvcam_get_frame**

```
int rvcam_get_frame(rvcam_hndl_t hndl,
                   rvcam_frame_info_t* p_frame_info,
                   uint64_t timeout,
                   unsigned int flags);
```

功能描述：调用 `rvcam_remote_get_frame` 获取一帧已采集的帧信息，获取后该帧标记为被消费。

- **`rvcam_release_frame`**

```
int rvcam_release_frame(rvcam_hndl_t hndl,
                       uint64_t idx);
```

功能描述：调用 `rvcam_remote_release_frame` 释放 `idx` 对应的 buffer，重新加入虚拟摄像头缓冲区队列。

6.2.3 共享 buffer

在 Android 单系统中，直接使用 Binder 传输 DMA buffer 即可。硬隔离双系统中，Automotive Camera HAL3 和 EVS HAL 需要共享 buffer 给 RVCAM 中间件，由于 Android 和 Linux 系统内存不共享，因此需要使用 CMA 分配连续的物理内存，再由 RPMSG heap 导出共享内存供系统间通信使用。

分配 CMA buffer 需要加上 `GRALLOC_USAGE_PRIVATE_3` 标志位

```
#define RK_GRALLOC_USAGE_PHY_CONTIG_BUFFER GRALLOC_USAGE_PRIVATE_3

#define GRALLOC_USAGE_PRIVATE_3 1ULL << 31
```

应用申请到 CMA buffer 后，client 端导出 RPMSG 共享内存的物理地址，经由 eRPC 传递给 server 端，server 端根据物理地址导出 RPMSG buffer fd，供 librvcam 轮转和 RGA 拷贝。

RPMSG 共享内存相关接口封装在 eRPC 中，初始化/去初始化接口如下：

```
rpmsg_mem_init();
rpmsg_mem_deinit();
```

导入共享内存相关接口使用过程如下：

```
// client 端
rpmsg_mem_info_t info{};
rpmsg_mem_import_from_dma_buf(dma_buf_fd, &rpmsg_mem_info);

// server 端
rpmsg_mem_info_t info{};
rpmsg_mem_import_from_phys_addr(phys_addr, size, &rpmsg_mem_info);
```

6.3 单系统与双系统编译和配置说明

6.3.1 编译选项

Camera Provider HAL3 和 EVS HAL 都动态连接 `librvcam_wrapper.so` 封装库，`rvcam_device.mk` 根据系统环境变量 `BOARD_ROCKCHIP_VEHICLE_MOS` 自动区分单系统还是双系统，以此定义 `persist.vendor.rockchip.rvcam.wrapper.version` 属性，`remote` 选项表示 `librvcam_wrapper.so` 动态加载 `librvcam_remote.so`，即使用 eRPC 传输层实现跨系统通信；`hidl` 选项表示 `librvcam_wrapper.so` 动态加载 `librvcam_client@1.0.so`，即使用 Binder 传输层在单系统上通信。

```
// rvcam_device.mk

ifeq ($(strip $(BOARD_ROCKCHIP_VEHICLE_MOS)),true)
    PRODUCT_PROPERTY_OVERRIDES +=
    persist.vendor.rockchip.rvcam.wrapper.version=remote
else
    PRODUCT_PROPERTY_OVERRIDES +=
    persist.vendor.rockchip.rvcam.wrapper.version=hidl
endif
```

6.3.2 Camera Provider HAL3 模式配置

`automotive_camera_config.xml` 中的 `HalBufferTransfer` 选项，仅在 Android 单系统中可以配置，硬隔离双系统默认使用 `false` 选项。

```
<HalBufferTransfer enabled="true"/>
```

7. FAQ

8. 缩略语

缩写	全称
RVCAM	Rockchip Vehicle Camera
SerDes	Serializer Deserializer
PoC	Power over Coex
FuSa	Function Safety
FMEA	Failure mode and effects analysis
EVS	Extended View System
DCM	Device Compatibility Matrixes
FCM	Framework Compatibility Matrixes
RPC	Remote Procedure Call
eRPC	Embedded RPC
DRM	Direct Rendering Manager
CMA	Contiguous Memory Allocator